MASARYK
UNIVERSITY

FACULTY OF INFORMATICS

A Comparative Study of Automated and Manual Creation of Test Management Plans and Test Cases

Master's Thesis

ABDUL BASIT

Brno, Fall 2025

MASARYK UNIVERSITY

FACULTY OF INFORMATICS

A Comparative Study of Automated and Manual Creation of Test Management Plans and Test Cases

Master's Thesis

ABDUL BASIT

Advisor: Prof. Ing. Tomáš Vojnar, Ph.D

Faculty of Informatics

Brno, Fall 2025


FACULTAS ARTIS INFORMATICÆ
UNIVERSITAS MASARYKIANA

Declaration

I hereby declare that this thesis is my original work, which I have written independently. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, I used the following AI tools: Claude AI for literature review synthesis, grammar checks, improving writing style, and as a research subject for experimental evaluation; ChatGPT, Microsoft Copilot, DeepSeek, and Grok as research subjects for the experimental assessment of test case generation capabilities; Google Gemini for data analysis and results interpretation. I confirm that I have used these tools while strictly following academic integrity guidelines. I have checked the content with reliable sources and my own experimental data, and I take full responsibility for the results.

Abdul Basit

Advisor: Prof. Ing. Tomáš Vojnar, Ph.D

Acknowledgements

I want to express my sincere gratitude to my supervisor, Prof. Ing. Tomáš Vojnar, PhD, for his expert guidance, insightful feedback, and ongoing support throughout this research project. His depth of knowledge in software testing, combined with his constructive suggestions, played a crucial role in shaping the direction and quality of this work.

I am also thankful for the valuable contributions of my consultant, Ing. Patrik Procházka, which significantly enhanced the experimental methodology of this study.

Finally, I would like to extend my heartfelt thanks to my family and friends for their unwavering support, motivation, and encouragement. Their belief in my abilities helped me persevere through the most challenging stages of this journey.

Abstract

This thesis evaluates five large language models (ChatGPT, Microsoft Copilot, DeepSeek, Grok, and Claude) through a two-phase experimental approach involving 32 controlled experiments. In the first phase, the models are evaluated on their ability to extract both functional and non-functional requirements from various document formats, including Software Requirements Specifications (SRS), user stories, and use case diagrams. Based on their performance in this phase, the top models advance to Phase 2, which focuses on evaluating the quality of test case generation. Phase 2 is divided into three stages: Stage 1 is an initial evaluation of all five models to rank their performance, Stage 2 compares the top three models (ChatGPT, Microsoft Copilot, and Claude) more systematically, eliminating the underperformers, and Stage 3 involves a detailed comparison between the best-performing AI model (Claude) and professional human testers using double-blind validation and statistical analysis. The results show that Claude outperformed humans in terms of accuracy, with significantly faster test case generation and perfect traceability of requirements. However, the AI models demonstrated lower completeness and struggled with contextual understanding. The study concludes that combining AI's speed and systematic test case generation with human expertise yields the best results, offering high accuracy, comprehensive coverage, and significantly greater efficiency.

Keywords

LLM, Software Engineering, Testing

Contents

List of Tables

Introduction

Software testing is an essential component of quality assurance within the software development lifecycle. However, it is also one of the most resource-intensive and time-consuming activities, with test case creation and maintenance representing a substantial share of the costs. The manual process of developing test cases not only imposes a financial burden but also introduces the risk of human error, particularly as software systems become more complex and development cycles shorten under competitive pressures.

The rise of large language models (LLMs) has opened up new opportunities for automating tasks traditionally performed by human experts. These models, trained on vast amounts of text and code, excel in understanding natural language, interpreting technical specifications, and generating structured documentation. Among their various applications in software engineering, automating test case generation from requirement specifications has emerged as a promising area. If generative AI can reliably produce high-quality test cases from Software Requirements Specification (SRS) documents, it could resolve long-standing bottlenecks in software development, potentially lowering costs, reducing time, and enhancing consistency in testing.

Despite the potential benefits, the practicality of AI-generated test cases remains uncertain. While these models have demonstrated impressive results in controlled environments, their performance in real-world scenarios where specifications are often complex, incomplete, ambiguous, or domain-specific requires further exploration. Furthermore, a thorough comparison between AI-generated and human-written test cases has not been fully explored. To assess whether generative AI can augment or even replace human testers, empirical evidence from rigorous comparative studies is essential.

This thesis investigates the strengths and weaknesses of five leading LLMs: ChatGPT, Microsoft Copilot, DeepSeek, Grok, and Claude in generating test cases from SRS documents. Through systematic comparison with human-authored test cases, this research provides empirical evidence on the practical utility of AI-assisted testing workflows.

The study employs a two-phase experimental approach:

- **Phase 1: Requirements Extraction Analysis** evaluates each model's fundamental capability to interpret and extract structured requirements from various types of documentation. This phase establishes baseline performance and identifies the strongest candidates for test case generation.
- **Phase 2: Test Case Generation and Evaluation** proceeds through three progressive stages:
  - Stage 1 conducts an exploratory assessment across all five models.
  - Stage 2 performs a systematic comparison of the top-performing models.
  - Stage 3 executes a comprehensive human vs AI evaluation, with statistical validation.

This progressive elimination strategy ensures a rigorous evaluation while maintaining experimental efficiency.

The research tackles the following key questions:

1. How accurately do AI models interpret software requirements?
2. What efficiency differences exist between AI-generated and human-authored test cases?
3. How well do the test cases cover the specified requirements in the source?
4. How effective are AI-generated test cases at detecting bugs?
5. What are the key challenges and limitations when using AI for test case generation?

Through fifteen controlled experiments comparing AI-generated and human-written test cases across multiple quality metrics, including accuracy, completeness, coverage, traceability, and efficiency, this study provides concrete evidence of where AI models excel and where they fall short compared to human testers.

The findings highlight the complementary strengths of both AI and human testing. AI models excel in speed and consistency, rapidly

generating comprehensive test suites with perfect traceability to requirements. However, they struggle with interpreting business logic, prioritizing risks, and handling exploratory testing scenarios. In contrast, human testers exhibit superior contextual understanding, creativity in negative testing, and the ability to assess risks in alignment with organizational priorities.

The research indicates that neither AI nor human testing achieves optimal results independently. Instead, hybrid approaches that combine AI-driven test generation with human validation deliver superior results across all quality dimensions.

This thesis contributes to the understanding of generative AI in software testing by providing a systematic comparative evaluation of five prominent LLMs, establishing a robust methodology for evaluating AI testing tools, and characterizing the complementarity between AI and human testers. It also offers practical guidance for organizations seeking to implement AI-assisted testing workflows and lays the foundation for future research on human-AI collaboration in testing.

The structure of the thesis is as follows: Chapter 2 outlines the background on software testing principles, requirements engineering practices, large language models, and related studies in AI-assisted software engineering. Chapter 3 describes the research methodology, including experimental design, evaluation criteria, and data collection procedures. Chapters 4 through 6 present the results from the three experimental phases: requirements extraction analysis, progressive model evaluation, and a comprehensive comparison of human and AI performance. Chapter 7 discusses the challenges and limitations encountered during the study. Finally, Chapter 8 concludes with a summary of the findings, practical recommendations, and suggestions for future research.

1 Background and Related Work

This chapter discusses the theoretical foundation of software testing, providing an understanding of the research investigation. It begins with the fundamentals of software testing and the crucial role of test cases in the quality assurance roadmap. The discussion then proceeds to requirements engineering practices, with a particular focus on Software Requirement Specification documents, which serve as the foundation for test case generation. Subsequently, the chapter explores large language models, their architectures, and capabilities relevant to natural language understanding and structured output generation. The chapter concludes with a review of related work in AI-assisted software engineering, identifying research gaps that motivate the current investigation.

1.1 Software Testing Fundamentals

Software testing is a vital activity in the software development lifecycle, designed to ensure that software functions correctly and meets user expectations. Testing spans several stages, including unit testing, integration testing, system testing, and acceptance testing. Each stage is crucial for detecting errors and ensuring that the system aligns with the requirements specified. Research consistently shows that identifying defects early in the development process is significantly more cost-effective than doing so later. This cost-benefit aspect encourages organizations to invest heavily in comprehensive testing strategies, though such strategies require substantial resources.

1.1.1 The Role of Testing in Software Development

Software testing involves several stages, each serving a unique purpose while contributing to the overall goal of ensuring quality. It starts with unit testing during the implementation phase, which checks individual components on their own. Integration testing follows, focusing on how different modules work together. System testing then assesses the entire system's functionality against the requirements, and finally,

acceptance testing ensures the system meets the expectations of stakeholders before deployment.

The financial impact of testing is significant. Research consistently shows that finding defects late in the development process is much more costly to fix than catching them early. This cost factor encourages organizations to invest heavily in thorough testing strategies, even though they require substantial resources.

1.1.2 Test Case Structure and Components

Test cases are the foundation of software testing, detailing the actions to be taken, expected results, and criteria for evaluating the software. They typically include identifiers, preconditions, test steps, test data, expected results, and pass/fail criteria. Adequate test case documentation is essential for ensuring clear communication between development teams and for regulatory compliance, especially in highly regulated industries. Following professional standards, such as IEEE 829, ensures that test cases remain usable across different stages of the software development process and can adapt to system updates.

1.1.3 Test Management and Planning

Effective test management goes beyond having well-documented test cases; it requires careful planning. A test management plan defines the overall strategy, resource allocation, risk management, and test scheduling. Requirements traceability matrices (RTM) are crucial for ensuring that all requirements are adequately tested. This tool links each test case to specific requirements, helping ensure complete test coverage and preventing unnecessary duplication.

1.2 Requirements Engineering

Requirements engineering is the process of defining and documenting what a system should do. The quality of the requirements documentation is critical for the development and testing phases. Software Requirements Specification (SRS) documents outline a system's functionality, constraints, and quality attributes. High-quality SRS docu-

ments are clear, complete, consistent, and verifiable, providing a solid foundation for both implementation and testing.

1.2.1 Software Requirements Specification Documents

As previously mentioned, Software Requirements Specification (SRS) documents provide formal descriptions of a system's functionality, constraints, and quality attributes. These documents serve as a blueprint for the entire software development process and are integral to generating test cases. The SRS must be complete, consistent, clear, verifiable, and traceable to ensure that it can be effectively used in both development and testing activities. If an SRS is poorly written, with vague or contradictory requirements, it can lead to inefficiencies in both the implementation and testing phases, affecting the overall quality of the software.

1.2.2 Functional and Non-Functional Requirements

Requirements are typically categorized as *functional* and *non-functional*:

- **Functional requirements** define what the system should do, describing its behavior, inputs, outputs, and processing logic. These requirements are often easier to translate into test cases as each functional requirement can correspond directly to a specific validation scenario.

- **Non-functional requirements**, on the other hand, describe how the system should perform, specifying constraints on system behavior such as performance, security, usability, and reliability. These are generally more challenging to test because they often require specialized tools and testing environments.

While functional requirements are more straightforward to implement and test, non-functional requirements often require advanced techniques and tools for testing, making them a critical area for AI models to address.

1.2.3 Requirement Quality and Testability

The *testability* of requirements is a key factor in their usefulness for generating effective test cases. Requirements must be:

- **Measurable**: They should be expressed in specific, quantifiable terms to allow for objective validation.
- **Deterministic**: They should define clear, unambiguous behaviors without leaving room for uncertainty.
- **Independent**: Each requirement should be verifiable on its own, without relying on the validation of other requirements.

Poorly written requirements can lead to ambiguous, incomplete, or inconsistent test cases, which can reduce the effectiveness of both manual and automated testing processes. Ensuring that requirements are testable is vital for enabling AI models to generate comprehensive and accurate test cases.

1.3 Large Language Models

Large language models are a type of artificial intelligence designed to process and generate natural language. These models are trained on vast amounts of text data, enabling them to understand and produce human language in a variety of contexts. Recent advancements in model design, training techniques, and computational power have led to systems that exhibit impressive abilities across a wide range of language-related tasks, from comprehension to generation.

1.3.1 Architecture and Fundamental Capabilities

Modern large language models primarily rely on transformer architectures, which were introduced by Vaswani et al. These models utilise attention mechanisms to handle sequential data, enabling them to process and comprehend language more efficiently. By being trained on billions of text samples, they learn statistical patterns in language, developing a deep understanding of grammar, meaning, common-sense reasoning, and even domain-specific knowledge.

The training process typically unfolds in two stages. Pre-training exposes the model to a broad range of text, helping it develop general language skills. Following this, fine-tuning or instruction-tuning refines the model's abilities for specific tasks or behaviours, usually by training it on more specialised datasets or incorporating human feedback. This two-phase approach enables models to acquire both broad knowledge and the specialised skills required for specific applications.

These models' capabilities go far beyond simple text completion. They excel in tasks like question answering, where they extract relevant information from the provided context or apply their learned knowledge. They can perform language translation between many different language pairs. They are also proficient in summarisation, condensing long texts into concise versions while retaining the key information. Their reasoning abilities allow them to solve complex, multi-step problems that require logical inference. In the context of software engineering, they can even understand and generate structured formats such as code, JSON, XML, and other machine-readable data representations.

1.3.2 Models Examined in This Research

This investigation evaluates five prominent large language models, each representing different approaches to model development and deployment:

ChatGPT is OpenAI's leading conversational AI, built on the GPT (Generative Pre-trained Transformer) architecture. Over time, successive versions have continually improved its capabilities, with recent iterations showing impressive performance across a wide range of natural language tasks. The model is particularly skilled at following complex instructions and generating clear, well-organized outputs that adhere to specified formats.

Microsoft Copilot builds on its partnership with OpenAI, while also aligning closely with Microsoft's enterprise focus. The system is designed to integrate seamlessly with development tools and enterprise software environments, offering significant benefits for professional workflows. Training and fine-tuning are likely tailored to enhance performance in areas like technical documentation and software engineering, making it a valuable asset in those contexts.

DeepSeek offers an alternative approach to developing large language models, possibly focusing on unique architectural innovations or specialised training strategies. The specific design choices and training methods used in DeepSeek shape its performance, particularly in tasks that involve generating structured outputs. These distinct features likely contribute to how well it handles tasks that require precision and organisation in its generated responses.

Grok represents a distinct developmental approach in the rapidly evolving field of large language models. Its unique training corpus, architectural design, and optimisation goals likely result in performance characteristics that set it apart from other models, excelling in specific tasks or applications while having distinct strengths or weaknesses in others. These differences contribute to its performance across various categories of functions.

Claude, developed by Anthropic, is designed with a strong emphasis on safety, helpfulness, and harmlessness. Its unique constitutional AI training approach is specifically aimed at aligning the model's behaviour with human values, minimising harmful outputs. This focus on ethical design gives Claude a distinct advantage in tasks that require careful reasoning and the generation of well-structured outputs. Additionally, the model is trained to avoid producing speculative or fabricated content, making it exceptionally reliable in scenarios where accuracy and safety are paramount.

1.3.3 Limitations and Challenges

Despite their impressive capabilities, large language models have several well-known limitations that can affect their usefulness in software testing. One of the most concerning issues is hallucination when models generate information that sounds plausible but is factually incorrect. In testing scenarios, this can lead to false test conditions, creating a misleading sense of confidence in the system's quality or directing validation efforts toward non-existent issues.

Another limitation is the context window, which restricts the amount of information a model can process at one time. While newer models have larger context capacities, long documents, such as Software Requirements Specifications (SRS), may still exceed these limits. This

often requires splitting documents into chunks, which can cause the model to lose necessary cross-references or context between sections.

Models also lack a proper understanding of domain-specific contexts. While they can identify common patterns and terminologies, they lack the depth of knowledge that domain experts possess regarding system architectures, business logic, or operational constraints. This can affect their ability to generate comprehensive test cases that account for implicit behaviours that may not be explicitly mentioned in the requirements.

Finally, consistency is another challenge. When given the same prompt, models may produce different outputs at different times. While this variability can be helpful in exploratory tasks, software testing requires reproducibility and deterministic behaviour. Unfortunately, statistical models like these can struggle to guarantee this level of consistency, which is critical for reliable testing.

1.4 AI in Software Engineering

The use of AI in software testing is gaining traction. While AI-assisted tools like GitHub Copilot and Tabnine have been used for code completion and bug detection, there is growing interest in automating test case generation. Traditional methods of test generation, such as random testing, symbolic execution, and search-based testing, are being complemented by AI-driven approaches that can produce more structured, consistent, and reusable test cases. However, the challenge lies in ensuring that AI-generated test cases accurately cover all requirements and align with human understanding of system behaviour.

1.4.1 Code Generation and Completion

AI-assisted code generation has become one of the most prominent and widely adopted applications of large language models in software engineering. Tools like GitHub Copilot, Amazon CodeWhisperer, and Tabnine offer real-time code suggestions by understanding the context of partially-written programs. These tools are especially effective for generating boilerplate code, implementing common algorithms, and converting natural language descriptions into functional code.

Research on the use of these tools has yielded mixed but generally positive results. Developers using AI code assistants often experience increased productivity, particularly for routine tasks that involve repetitive coding. However, the quality of the generated code can be inconsistent. AI-generated suggestions typically require thorough review and may need adjustments before being integrated into production systems. One significant concern is security vulnerabilities; since these models are trained on vast datasets, they can unintentionally suggest insecure coding practices or patterns learned from flawed sources. This underscores the need for careful oversight when using AI in coding tasks.

1.4.2 Documentation and Requirements Processing

Natural language processing (NLP) is increasingly being applied to various aspects of software engineering, particularly in documentation tasks. Systems are now able to generate API documentation directly from code, translate between natural and formal specifications, and summarise complex codebases. Recent research has also explored the use of language models to identify ambiguities or inconsistencies in requirements documents, which could improve the quality of specifications before implementation begins. One active research area closely related to this thesis is automated requirements extraction, the process of identifying and categorising requirements from natural language descriptions. Traditionally, this task relied on rule-based NLP, ontology-based reasoning, or supervised machine learning trained on annotated datasets. However, large language models offer distinct advantages, as they can understand context and manage linguistic variability without requiring extensive domain-specific customisation. This flexibility makes them a promising tool for extracting and processing requirements more efficiently and accurately.

1.4.3 Test Generation Approaches

Automated test generation has been a longstanding area of research, with several traditional methodologies in use. These include random testing, which generates inputs using stochastic processes; symbolic execution, which systematically explores program paths; search-based

techniques, which evolve test suites using genetic algorithms; and model-based testing, which derives tests from formal specifications.

In recent years, the application of large language models to test generation has gained attention. Some studies focus on generating unit tests from method signatures and code implementations, while others explore the creation of integration tests or end-to-end test scenarios. Despite these advances, comprehensive comparative evaluations, particularly those comparing AI-generated test cases with those created by humans, remain limited. This is especially true for system-level testing derived from requirement specifications rather than purely from code analysis, an area that remains underexplored.

1.5 Related Work

Recent research has explored various approaches to *automated test case generation* from natural language requirements using both traditional and AI-based methods.

- **Lafi et al. (2021)** combined rule-based NLP techniques with transformations to generate test cases from requirements. While their method showed feasibility, it required significant manual effort for rule creation and lacked flexibility in handling linguistic variations.

- **Allala et al. (2022)** focused on abstract test case generation using a combination of model-driven engineering and NLP. Their approach required constructing formal models from user requirements, which, while systematic, involved high upfront investment in model creation, making it less adaptable for quick use.

- **Wang (2020)** demonstrated an NLP-based technique for generating acceptance test cases from use case specifications. However, the method struggled with less structured requirements, highlighting the challenges of applying automated test generation across varied document formats.

- **Haldar et al. (2024)** explored the use of *ChatGPT* in test planning and generation. While the results were promising, concerns

over *hallucinations* and the need for human oversight were emphasized. Their work primarily focused on preparatory testing activities but did not conduct a detailed comparison with human-generated test cases.

- **Arora et al. (2024)** investigated the use of retrieval-augmented LLMs for generating test scenarios. Their study showed that augmenting models with relevant context significantly improved the quality of test scenarios. However, the evaluation primarily focused on test scenario outlines, leaving a gap in the full test case generation process.

- **Chenail-Larcher et al. (2025)** compared AI-based and hybrid approaches for test generation from use case specifications in IoT systems. Their hybrid method, combining traditional techniques with LLMs, outperformed both approaches individually, suggesting that combining AI with other methods might yield better results.

Despite these advancements, comprehensive comparative studies between AI-generated and human-written test cases across various quality dimensions are limited. Much of the existing research focuses on demonstrating the feasibility of automated approaches or evaluating specific attributes like coverage, but a broader evaluation of quality, maintainability, and effectiveness in real-world scenarios is still lacking.

2 Research Methodology

This chapter outlines the systematic approach used to investigate the capabilities of generative AI in generating test cases. The research follows a multi-phase experimental design that integrates both quantitative measurements and qualitative analysis to provide a thorough evaluation. A key feature of this methodology is the dual-perspective evaluation, where the researcher's independent analysis is complemented by LLM-assisted verification. This ensures a comprehensive assessment of AI performance while maintaining high standards of analytical rigour throughout the study.

2.1 Research Design Overview

The investigation employs a progressive experimental methodology designed to systematically evaluate large language model capabilities while building toward comprehensive comparison with human-generated test cases.

2.1.1 Two-Phase Progressive Structure

This research employs a two-phase progressive evaluation framework designed to systematically assess and compare the capabilities of five generative AI models in software testing tasks.

Phase 1: Requirements Extraction Analysis establishes each model's foundational capability to interpret software requirements documentation. Seven experiments evaluate how effectively each model extracts functional and non-functional requirements from various document formats, including formal SRS documents, user stories, and use case diagrams. Performance in this phase determines which models advance to test case generation evaluation.

Phase 2: Test Case Generation and Evaluation employs a three-stage progressive methodology:

- **Stage 1: Exploratory Five-Model Assessment** conducts initial evaluation of all five models' test case generation capabilities, establishing preliminary performance rankings.

- **Stage 2: Systematic Three-Model Comparison** focuses on the top three performers (ChatGPT, Microsoft Copilot, Claude), applying rigorous evaluation criteria to identify the strongest model while eliminating underperformers.

- **Stage 3: Comprehensive Human vs AI Evaluation** performs detailed comparison between the best-performing AI model and professional human testers across 15 controlled experiments, employing double-blind validation and statistical significance testing.

This progressive elimination strategy balances thoroughness with efficiency, allowing deep investigation of the most promising models while maintaining experimental rigor throughout.

2.1.2 Model Selection and Configuration

The research focuses on five prominent large language models: ChatGPT (OpenAI), Microsoft Copilot, DeepSeek, Grok, and Claude (Anthropic). These models were selected based on their current availability, documented capabilities in structured output generation, and representation of diverse development approaches. Claude, which underwent the most extensive evaluation, had its temperature fixed at 0.0 and top-p set to 1.0 to maximise output determinism and reproducibility. Additionally, fixed random seeds were used to ensure consistency across all experimental trials, allowing for reliable comparisons between the models.

2.1.3 Experimental Artifact

The CORADS (Car On Roads Advertisement) Software Requirements Specification serves as the primary experimental artefact for this research. This document outlines the specifications for a location-based mobile advertising platform, featuring multiple user roles, real-time tracking, payment processing, and campaign management functionality. Several key characteristics made CORADS particularly suitable for this study: it adheres to professional quality and IEEE standards, offers comprehensive coverage of both functional and non-functional

requirements, and presents a realistic domain complexity. Additionally, the document is sufficiently complete to support meaningful test generation while containing inherent ambiguities that help reveal the limitations of the models being evaluated.

2.2 Phase 1: Requirements Extraction Evaluation

Phase 1 establishes fundamental capabilities by evaluating how effectively each model extracts and categorizes requirements from specification documents. Accurate requirement comprehension constitutes a prerequisite for meaningful test case generation.

2.2.1 Experimental Procedure

Each model received identical prompts instructing comprehensive extraction with proper categorization and structured output formatting. The extraction task demands both linguistic understanding and domain knowledge, as requirements appear distributed throughout specifications intermixed with contextual information, examples, and design discussions.

2.2.2 Evaluation Metrics

The six main metrics used to evaluate the quality of requirement extraction are:

Coverage measures the percentage of actual requirements that are successfully identified against a manually derived ground truth reference set.

Clarity evaluates the readability and precision of extracted formulations using a qualitative rubric that considers linguistic quality, technical accuracy, and stakeholder accessibility.

Traceability examines whether extractions maintain clear relationships to source material through unique identifiers, section references, and hierarchical organisation.

Completeness assesses inclusion of all requirement types, including business rules, legal requirements, testing considerations, and quality attributes.

Redundancy identifies duplicate or unnecessarily overlapping requirements. Lower scores indicate better performance.

Hallucination detection identifies fabricated requirements lacking a foundation in source documentation, which is the most critical concern, as hallucinated requirements could mislead development efforts.

2.3 Phase 2: Test Case Generation Evaluation

Phase 2 systematically evaluates test case generation capabilities through two progressive stages with models selected from Phase 1.

2.3.1 Stage 1: Exploratory Five-Model Assessment

Experimental Procedure

Stage 1 conducts initial exploratory assessment of all five models' test case generation capabilities. Each model receives identical SRS excerpts and is prompted to generate comprehensive test case suites.

Experiments: 3 exploratory experiments **Models Evaluated:** ChatGPT, Microsoft Copilot, DeepSeek, Grok, Claude **Evaluation Focus:** Overall structure, coverage breadth, traceability, and professional formatting

Evaluation Approach

Given the exploratory nature of Stage 1, evaluation emphasized qualitative assessment of:

- Test case structure and organization
- Requirements coverage breadth
- Traceability to source requirements
- Professional formatting and clarity
- Practical usability

2.3.2 Stage 2: Systematic Three-Model Comparison

Experimental Design

Stage 2 performs rigorous systematic comparison of the three top performers identified in Stage 1.

Experiments: 4 controlled experiments **Models Evaluated:** ChatGPT, Microsoft Copilot, Claude **Evaluation Focus:** Quantitative assessment across standardized metrics

Evaluation Metrics

Each model's output is scored on a 100-point scale across seven dimensions:

- Accuracy (alignment with SRS requirements)
- Completeness (inclusion of all required test case fields)
- Test Coverage (functional and non-functional requirements)
- Efficiency (time and effort required)
- Quality (detail, specificity, maintainability)
- Bug Detection capability
- Usability (automation readiness)

Stage 2 Results and Model Selection

Comparative analysis revealed Claude and Microsoft Copilot as the strongest performers, with Claude demonstrating superior completeness and coverage of non-functional requirements. ChatGPT, while competent, showed limitations in depth and breadth, particularly for complex scenarios. Based on these results, Claude was selected for comprehensive human comparison in Stage 3.

2.3.3 Stage 3: Comprehensive Human vs AI Evaluation

Experimental Design

Stage 3 conducts rigorous comparison between Claude (the best-performing AI model) and professional human testers.

Experiments: 15 controlled experiments **Comparison:** Claude AI vs professional quality assurance engineers **Validation Method:** Double-blind human evaluation with inter-rater reliability assessment

Expanded Evaluation Framework

Stage 3 employs an expanded evaluation framework measuring:

- Accuracy and hallucination control
- Completeness and contextual understanding
- Requirements traceability (RTM coverage)
- Oracle quality and measurability
- Risk prioritization and business judgment
- Test suite structure and redundancy
- Efficiency and productivity

Progressive Prompt Refinement Strategy

The approach to prompt engineering was gradually refined over several iterations to improve the performance of the AI models. This process involved learning how the prompts were structured, the clarity of the instructions, and the specific roles assigned to the AI, which influenced the quality of the results.

Statistical Validation

All quantitative comparisons employ appropriate statistical tests:

- Paired t-tests for continuous metrics

- Wilcoxon signed-rank tests for non-parametric data
- Cohen's d for effect size calculation
- Cohen's kappa for inter-rater reliability

2.4 Dual-Perspective Analytical Approach

A distinguishing methodological feature involves dual-perspective analysis combining the researcher's independent evaluation with LLM-assisted verification.

2.4.1 Researcher's Primary Analysis

For each experiment, I performed a comprehensive analysis, leveraging expertise in the testing domain and knowledge of quality assessment. This analysis utilised structured rubrics, while also incorporating qualitative judgment on factors like appropriateness, practical usability, and alignment with professional standards. The researcher systematically compared the test cases across all metrics, documenting specific strengths, weaknesses, and recurring patterns. Human-led analysis was crucial for several reasons: software testing requires a contextual understanding, recognising when test cases that appear correct actually miss critical scenarios, identifying subtle quality differences, and evaluating the practical utility of test cases in real-world workflows. The researcher closely examined each test case, uncovering patterns in model behaviour, the characteristic manifestation of hallucinations, and the systematic differences between AI-generated and human-generated strategies.

2.4.2 LLM-Assisted Verification

Following primary analysis, a large language model received prompts requesting independent evaluation of the same test case sets. This secondary analysis served multiple purposes:

Bias Mitigation: The researcher naturally brings expectations and potential biases. LLM analysis, operating under different assumptions, may identify overlooked aspects.

Consistency Verification: When AI and human assessments substantially agree, confidence in conclusions increases. Disagreements prompt re-examination.

Alternative Perspective: LLMs may recognise patterns or relationships less apparent to human reviewers through statistical analysis of training data.

Detailed Documentation: LLM analysis generates extensive, structured documentation that enriches the research record. Verification prompts maintained independence by not revealing the researcher's conclusions. Instead, they provided test cases, evaluation criteria definitions, and requested a systematic assessment with justification.

2.4.3 Synthesis Process

The dual-perspective approach culminated in a synthesis of the assessments, where areas of strong agreement were accepted without further scrutiny. Disagreements prompted a deeper investigation of specific test cases or evaluation dimensions. Ultimately, the researcher's evaluation took precedence for final decisions, as human judgment is considered the authoritative factor in software testing contexts. However, disagreements between the researcher and the LLM often highlighted legitimate ambiguities, prompting more careful consideration and improving the overall quality of the analysis. This methodology strengthens the research's credibility by recognising the value of both human expertise and AI's analytical contributions, positioning AI as an aid in analysis and verification, rather than as an autonomous decision-maker.

2.5 Statistical Analysis Methods

Statistical analysis establishes confidence in quantitative findings and determines whether observed differences achieve significance.

2.5.1 Inter-Rater Reliability

Cohen's kappa coefficient was used to measure the agreement between evaluators for subjective assessments that required judgment, such as

quality ratings (on a 1-5 scale) and priority classifications (High/Medium/Low). The research aimed for a kappa score of 0.70, indicating substantial agreement. When preliminary assessments yielded lower kappa values, evaluators discussed the criteria, clarified any ambiguities, and reassessed the cases until an adequate level of agreement was reached. This process ensured that subjective judgments were consistent and reliable across evaluations.

2.5.2 Significance and Effect Size

Paired t-tests and Wilcoxon signed-rank tests were used to assess whether performance differences between models were statistically significant, with a threshold set at $p < 0.05$. In addition to determining significance, Cohen's d was used to quantify the magnitude of these differences. The interpretation of effect size follows conventional guidelines: 0.2 represents small effects, 0.5 represents medium effects, and 0.8+ indicates large effects. The research reports both the statistical significance and effect size for key comparisons to provide a more comprehensive understanding of the models' performance differences.

2.6 Methodological Limitations

The research acknowledges several inherent limitations. The focus on a single SRS document ensures necessary experimental control but limits the generalizability of the findings across different domains. The model version temporality reflects the capabilities of the models at the time of the investigation; newer releases may exhibit different performance characteristics. Despite the use of a dual-perspective analysis and statistical validation, some subjectivity in evaluation remains, introducing potential bias. However, the systematic methodology helps mitigate this risk; it cannot eliminate it. These limitations are common to all empirical AI research and do not undermine the validity of the findings concerning the evaluated model versions.

3 Requirements Extraction Analysis

This chapter presents the findings from Phase 1 experiments, which evaluate the capabilities of five large language models in extracting and categorising requirements from Software Requirements Specification (SRS) documents. This phase establishes the fundamental competencies required for effective test case generation by conducting a comprehensive assessment across several evaluation dimensions. The results identify the top-performing models, which are deemed capable of progressing to Phase 2, where they will undergo further evaluation in test case generation.

3.1 Experimental Setup

Phase 1 involved a series of experiments designed to assess the extraction capabilities of five large language models across various contexts using diverse requirement documents. In each experiment, models were provided with complete SRS documents or requirement specifications in multiple formats, along with instructions to extract both functional and non-functional requirements, ensuring proper categorization and structured organisation. The experiments employed a variety of prompting strategies to evaluate the models' robustness. Initial prompts were straightforward, but subsequent trials refined the structure of the prompts, corrected grammatical elements, and sometimes incorporated deep thinking techniques to encourage more sophisticated analysis. This variation allowed the research to determine whether model performance was primarily influenced by intrinsic capabilities or its sensitivity to prompt engineering. The evaluation was based on six primary metrics, each scored on a normalized 100-point scale: Coverage (percentage of requirements successfully identified), Clarity (readability and precision of formulations), Traceability (clear linkage to source material), Completeness (inclusion of all requirement types), Redundancy (absence of unnecessary duplication, inversely scored), and Hallucination (degree of fabricated content, inversely scored). Each of these metrics was independently assessed through a systematic comparison against manually derived

ground truth reference sets, ensuring a comprehensive and objective evaluation of the models' performance.

3.2 Overall Performance Summary

Analysis across all Phase 1 experiments reveals consistent performance patterns. Table 4.1 presents aggregated scores averaged across experiments, providing overall performance assessment for each model.

Table 3.1: Phase 1 Aggregate Performance Scores (0–100 scale)

Model	Coverage	Clarity	Traceability	Completeness	Redundancy	Hallucination	Average
Claude AI	97.5	90.0	95.0	93.3	82.5	98.0	93.0
Microsoft Copilot	88.3	88.8	84.0	81.7	86.3	98.0	87.8
DeepSeek	88.8	84.4	87.5	86.7	90.3	97.0	88.9
ChatGPT	77.6	84.0	73.8	73.8	87.5	95.0	81.9
Grok AI	83.8	78.1	76.3	77.5	80.5	95.0	81.9

Claude AI achieved the highest overall performance with a score of 93.0, excelling in Coverage (97.5), Traceability (95.0), and Completeness (93.3). Microsoft Copilot and DeepSeek performed similarly, with scores of 87.8 and 88.9, respectively. DeepSeek stood out for its strong performance in Redundancy avoidance (90.3), while Copilot displayed balanced performance across all metrics. On the other hand, ChatGPT and Grok AI scored substantially lower, both at 81.9. ChatGPT demonstrated particular weaknesses in Coverage (77.6

3.3 Detailed Analysis by Metric

3.3.1 Coverage

Coverage measures how successfully the models identify requirements from the source documents. Claude AI achieved the highest coverage score (97.5), capturing nearly all functional and non-functional requirements, including edge cases and implicit specifications. It was particularly effective in identifying requirements spread across different sections of the document and in recognizing non-functional requirements, even when expressed indirectly through discussions of quality attributes. DeepSeek (88.8) and Microsoft Copilot (88.3) also performed well in terms of coverage, though they occasionally

missed requirements embedded in contextual discussions. DeepSeek tended to over-consolidate, merging distinct requirements into single entries, while Copilot sometimes overlooked specialized categories, such as testing considerations and legal compliance. Both ChatGPT (77.6) and Grok AI (83.8) exhibited significant limitations in coverage, frequently omitting business rules, assumptions, constraints, and specialised non-functional attributes, which negatively impacted their overall performance in requirement identification.

3.3.2 Clarity

Microsoft Copilot achieved the highest clarity score (88.8), producing well-structured outputs with precise categorisation and appropriate technical precision. It struck a balance between conciseness and the necessary level of detail, making the requirements easy to understand. Claude AI (90.0) and ChatGPT (84.0) also produced precise formulations, though Claude occasionally included unnecessary verbosity, while ChatGPT had some minor clarity issues in its wording. DeepSeek (84.4) generated readable outputs, but sometimes fragmented requirements in ways that could have been more cohesive. Grok AI scored the lowest in clarity (78.1), with outputs that often lacked consistent structure and included vague terminology, such as "reasonable time frame" and "appropriate security measures," which compromised the precision and clarity of the requirements.

3.3.3 Traceability

Claude AI excelled in traceability, achieving a score of 95.0, by providing explicit section references, page numbers, and detailed hierarchical traceability matrices that directly mapped requirements to their source locations. DeepSeek (87.5) also demonstrated strong traceability, offering clear attribution; however, its mappings were less detailed than those of Claude. Microsoft Copilot (84.0) provided adequate traceability, using identifiers and general references to link requirements, though with less precision. In contrast, ChatGPT (73.8) and Grok AI (76.3) showed significant weaknesses in traceability, with vague or missing source references. This lack of clear traceability made it more

challenging to verify the accuracy and completeness of the extracted requirements.

3.3.4 Completeness

Claude AI achieved the highest completeness score (93.3), systematically capturing all requirement categories and explicitly noting any ambiguities or areas that required further clarification. DeepSeek (86.7) provided reasonably complete extractions but occasionally missed specialised categories, such as audit logging and compliance requirements. Microsoft Copilot (81.7) sometimes overlooks important aspects, including business rules, assumptions, and constraints. ChatGPT (73.8) and Grok AI (77.5) showed notable gaps in completeness, frequently omitting crucial non-functional categories, business rules, and contextual constraints, which impacted their ability to capture the full scope of the requirements.

3.3.5 Redundancy

DeepSeek achieved the best redundancy control, scoring 90.3, and produced concise extractions with minimal duplication. ChatGPT (87.5) and Microsoft Copilot (86.3) demonstrated reasonable control over redundancy, although occasional overlaps in their outputs were noted. Grok AI (80.5) exhibited moderate redundancy, particularly in business rules and role-specific descriptions, which affected the clarity of its extractions. Claude AI scored the lowest in this category (82.5), occasionally repeating sub-requirements or presenting overlapping specifications. While this ensured thorough coverage, it led to an increase in documentation volume, which could complicate the overall clarity and efficiency of the extracted requirements.

3.3.6 Hallucination

All models demonstrated low hallucination rates. Claude AI (98.0), Microsoft Copilot (98.0), and DeepSeek (97.0) exhibited strong discipline in grounding extractions in actual content, typically flagging uncertainties explicitly. ChatGPT (95.0) and Grok AI (95.0) showed slightly elevated tendencies, occasionally generating requirements in-

ferred from domain knowledge rather than document content, risking discrepancies between documented intentions and extractions.

3.4 Critical Observations

Several cross-cutting observations emerged from the Phase 1 analysis. All models struggled with error-handling use cases, often missing requirements related to failure scenarios, exception processing, and recovery procedures. This consistent gap suggests that error conditions are insufficiently emphasised in standard requirement formulations, indicating the need for explicit attention to these aspects during both specification and extraction activities.

Compliance requirements also posed challenges across the models. While Microsoft Copilot occasionally identified requirements such as GDPR anonymisation or PCI-DSS standards, other models frequently overlooked necessary legal, regulatory, and compliance specifications. This shortcoming is particularly concerning in domains where compliance is critical.

Ambiguity resolution varied widely across the models. Claude AI consistently excelled in identifying and flagging ambiguous specifications, raising clarifying questions or noting multiple possible interpretations. In contrast, other models typically select a single interpretation without acknowledging uncertainty, which can lead to misalignment with stakeholder intentions.

The prompting strategy had an impact on model performance, though to varying degrees. All models performed better with structured, detailed prompts compared to more casual instructions. However, differences in intrinsic capabilities were more significant than variations in prompting; poor-performing models remained ineffective despite optimised prompts, while strong models maintained high performance even with suboptimal instructions.

3.5 Limitations Identified in Requirements Extraction

3.5.1 Visual and Non-Textual Input Limitations

A significant limitation was identified during experiments involving non-textual requirement sources, such as diagrams, use case visualisations, and class diagrams. All the AI models tested showed significantly reduced performance when tasked with processing visual information, compared to their ability to extract requirements from text-based sources. This limitation highlights the challenges AI faces when dealing with non-textual data, which requires a deeper level of understanding beyond simple textual analysis.

Diagram Processing Challenges

When the AI models were tasked with processing use case diagrams, UML class diagrams, and system architecture visualisations, they encountered several specific issues:

Text Extraction Without Context: While the models were able to extract text labels and annotations from the diagrams, they struggled to understand the structural relationships, like arrows, hierarchies, and spatial arrangements, which are crucial for interpreting the full meaning of the diagram. For example, in use case diagrams, the models could identify the actor names and use case labels, but they had difficulty understanding the "extends" and "includes" relationships between the use cases.

Relationship Misinterpretation: Class diagrams presented particular difficulties. Although the models could identify class names and occasionally method signatures, they often failed to interpret more complex relationships, like multiplicities, inheritance, and associations. For instance, a one-to-many relationship between entities like "Campaign" and "Advertisement" was often either missed or misunderstood.

Complete Visual Omission: In several cases, the models completely ignored diagrams when there was also accompanying text. This suggests a bias in the AI models, where they favored text-based

data over visual data, possibly due to how they were trained or optimized for textual input.

This visual processing gap is a significant limitation for workflows that rely on diagrammatic modelling languages, such as UML, SysML, or BPMN, for requirements engineering. Organisations using these visual specifications will need to either manually interpret diagrams or develop specialised diagram-to-text conversion preprocessing to supplement AI-based requirements extraction.

3.6 Model Selection for Phase 2

Based on Phase 1 performance, three models advanced to Phase 2 for test case generation evaluation: Claude AI, Microsoft Copilot, and ChatGPT. The selection criteria focused on overall performance, comprehension of requirements, and the practical utility of the models for testing workflows.

Claude AI emerged as the top performer, achieving the highest overall scores with exceptional strengths in coverage, traceability, and completeness. Its strong requirement extraction capabilities indicated a solid foundation for generating test cases that require a deep understanding of requirements.

Microsoft Copilot was selected for its balanced performance across all metrics, with particular strength in clarity. Its ability to produce well-structured, actionable outputs made it a promising candidate for generating usable test specifications, indicating potential for high-quality test case generation.

ChatGPT, despite lower scores in Phase 1, was included due to its widespread adoption and baseline competency. Phase 2 would determine if its limitations in requirement extraction hindered its test generation capabilities or if it could compensate for these weaknesses through other strengths.

DeepSeek and Grok AI did not progress to Phase 2, despite DeepSeek having a competitive overall score. DeepSeek's tendency toward over-consolidation and occasional gaps in completeness raised concerns about its ability to generate comprehensive test cases. Grok AI's weaknesses in clarity and traceability suggested fundamental limitations that would likely affect the quality of its test specifications. Due to

resource constraints, the focus in Phase 2 was placed on the most promising candidates.

4 Test Case Generation Evaluation

Building upon the requirements extraction capabilities established in Phase 1, Phase 2 evaluates the models' ability to generate high-quality test cases through a three-stage progressive evaluation framework.

4.1 Exploratory Five-Model Assessment

Stage 1 involved exploratory experiments that evaluated the test case generation capabilities of all five models: ChatGPT, Microsoft Copilot, DeepSeek, Grok, and Claude, which were previously assessed in the requirements extraction phase. These experiments utilised a range of prompting strategies and testing scenarios to evaluate each model's fundamental competencies in generating test cases. The goal was to establish baseline performance across the models, identifying their strengths and weaknesses in the test case generation process.

4.1.1 Experimental Approach

Stage 1 experiments varied the complexity and specificity of the prompts to evaluate the models' robustness in generating test cases. The initial experiments employed simple prompts that requested basic happy-path test cases. As the experiments progressed, the prompts became more sophisticated, asking not only positive and negative scenarios but also boundary value testing and comprehensive coverage of both functional and non-functional requirements. The advanced experiments focused on generating requirements traceability matrices, using formal test design techniques, and ensuring IEEE 829 compliance.

This variation in methodology aimed to determine whether the models required extensive prompt engineering to produce acceptable performance or if they demonstrated intrinsic test generation capabilities that remained resilient to variations in prompt quality. The results highlighted the extent to which the models could adapt to more complex testing scenarios without needing fine-tuned instructions.

4.1.2 Aggregated Performance Analysis

Table 5.1 presents Phase 1 aggregate performance across experiments and prompting strategies.

Table 4.1: Phase 1 Average Performance Scores

Model	Overall Score	Key Strengths	Primary Weaknesses
Microsoft Copilot	91.4	Technical precision, actionable steps, NFR coverage	Occasional breadth limitations
Claude AI	91.2	Comprehensive coverage, formal structure, RTM	Slight verbosity
ChatGPT	88.3	Clear presentation, traceability	Limited NFR depth
Grok	77.4	Modular organization	Weak coverage, minimal negatives
DeepSeek	–	Weakest – Consistently shallow outputs	–

Microsoft Copilot and Claude emerged as the top performers, with nearly identical overall scores (91.4 and 91.2, respectively). Both demonstrated professional-quality test case generation, each with distinct complementary strengths. ChatGPT achieved respectable performance (88.3), but showed notable gaps in non-functional requirement coverage and integration testing. Grok and DeepSeek scored substantially lower, revealing fundamental limitations in test comprehensiveness and depth, which hindered their ability to generate high-quality, thorough test cases.

4.1.3 Detailed Findings by Model

Microsoft Copilot demonstrated an exceptional balance between functional and non-functional requirement coverage, excelling in generating technically precise test specifications with clear validation criteria. Test cases included specific elements like HTTP status codes, API response formats, database state expectations, and quantified performance thresholds. Copilot effectively utilized formal test design

techniques such as boundary value analysis, equivalence partitioning, decision tables, and state transition testing. It gave particular attention to security testing, generating explicit scenarios for issues like authentication bypass, SQL injection, cross-site scripting, and data exposure vulnerabilities.

Claude AI produced the broadest test coverage, with systematic requirement traceability. It generated comprehensive test suites that spanned all functional areas, clearly mapping test cases back to SRS requirements using formal traceability matrices. The test case structure adhered closely to IEEE 829 standards, including complete preconditions, detailed test steps, specific test data, measurable expected results, and clear pass/fail criteria. Claude showed particular strength in identifying edge cases and covering negative scenarios, systematically exploring potential failure modes and error conditions.

ChatGPT generated well-structured test cases with clear presentation and logical organization. The model provided good functional coverage and often included explicit requirement mappings. However, its coverage of non-functional requirements was less comprehensive than that of the leading models. Security scenarios, performance validation, and integration testing received insufficient attention, and test data specifications were sometimes generic rather than providing concrete values.

Grok demonstrated basic functional test generation capability but lacked depth and comprehensiveness. Its test cases were mostly focused on happy-path scenarios with minimal negative testing. Non-functional requirements received minimal coverage, and the requirements traceability was weak, with vague or missing source references. The modular organizational structure provided some usability, but it couldn't compensate for the fundamental coverage gaps.

DeepSeek consistently produced the weakest outputs across the Phase 1 experiments. Its test cases exhibited narrow functional coverage, minimal attention to non-functional requirements, and insufficient detail for practical execution. The model's outputs required substantial human supplementation to reach acceptable quality, raising concerns about its utility in professional testing workflows.

4.1.4 Conclusions and Model Selection

Phase 1 established a clear performance hierarchy among the evaluated models. Microsoft Copilot and Claude demonstrated professional-level test generation capabilities, making them prime candidates for detailed evaluation in subsequent phases. ChatGPT achieved acceptable baseline performance, justifying its inclusion in Phase 2 to determine whether its limitations in requirement extraction constrained its test generation capabilities or if it could compensate through other strengths. On the other hand, Grok and DeepSeek exhibited fundamental limitations in their outputs, preventing them from advancing to the more systematic evaluation phases.

4.2 Systematic Three-Model Comparison

Stage 2 involved ten controlled experiments that systematically evaluated the three models selected from Phase 1: Microsoft Copilot, Claude, and ChatGPT. These experiments employed a consistent prompting methodology and applied the same evaluation criteria across various CORADS modules and testing scenarios. The aim was to assess each model's performance in a more structured and focused manner, allowing for detailed comparisons and insights into their test case generation capabilities.

4.2.1 Experimental Framework

All Stage 2 experiments used identical, structured prompts framed with a persona-based approach: "Act as a Principal Quality Engineer with 20+ years of experience." The prompts specified an IEEE 829-compliant test case structure, which included elements such as test ID, description, preconditions, test steps, test data, expected results, post-conditions, pass/fail criteria, priority, and test environment. Special emphasis was placed on ensuring comprehensive coverage, including edge cases, negative scenarios, security testing, and performance validation. This consistent approach ensured a thorough and standardised evaluation across all models.

Evaluation employed seven criteria scored on 100-point scales:

- **Accuracy:** Alignment with SRS requirements

- **Completeness:** Inclusion of all required test case components
- **Test Coverage:** Breadth across functional and non-functional requirements
- **Efficiency:** Appropriate test case count avoiding redundancy
- **Quality:** Logical clarity and execution detail
- **Bug Detection:** Effectiveness in revealing potential defects
- **Usability:** Clarity for manual and automated testing

4.2.2 Initial Three-Model Comparison

The first four experiments evaluated all three models across core CORADS modules to establish relative performance and identify candidates for elimination.

Table 4.2: Phase 2 Experiments 1–4 Results

Experiment	Module	Claude AI	Microsoft Copilot	ChatGPT	Winner
Exp 1	Driver Registration	94.3	89.3	75.7	Claude
Exp 2	Campaign Management	100.0	98.6	64.3	Claude
Exp 3	Payment System	98.6	92.9	70.0	Claude
Exp 4	Real-Time Tracking	96.4	92.9	82.1	Claude
Average	–	**97.3**	**93.4**	**73.0**	Claude

Claude achieved the highest scores across all four experiments, with an average of 97.3, demonstrating exceptional performance in test case generation. Microsoft Copilot also performed strongly, with a consistent average of 93.4, showcasing reliable capabilities across the evaluation criteria. In contrast, ChatGPT scored substantially lower, with an average of 73.0, exhibiting significant limitations in test coverage, particularly in non-functional requirements. It also struggled with completeness, often missing essential components in test cases, and had a weaker bug detection capability, as it tended to focus on

happy paths and lacked sufficient coverage of edge cases and failure scenarios.

ChatGPT Elimination: After Experiment 4, ChatGPT was formally eliminated from further Phase 2 evaluation. Its consistent underperformance across critical dimensions suggested fundamental limitations that could not be addressed through prompt engineering alone. Key issues included significant coverage deficiencies (20-30 points below those of the leading models), weaknesses in non-functional requirements (with minimal security and performance testing), structural limitations (incomplete test case components), and a lack of sufficient depth (generic specifications lacking concrete details). These limitations justified its elimination, allowing the research team to focus on the demonstrably superior models.

4.3 Focused Two-Model Evaluation

Experiments 5 through 10 compared Claude and Microsoft Copilot exclusively, exploring specialized testing scenarios and additional CORADS functionality.

Table 4.3: Phase 2 Experiments 5–10 Results

Exp	Module	Claude AI	Microsoft Copilot	Winner
Exp 5	Driver Registration (Security)	80.0	86.7	Copilot
Exp 6	Campaign Management (Advanced)	86.7	93.3	Copilot
Exp 7	Payment System (Integration)	90.0	77.0	Claude
Exp 8	Real-Time Tracking (Performance)	84.0	90.0	Copilot
Exp 9	Security & Edge Cases	90.0	83.0	Claude
Exp 10	Comprehensive (All Modules)	92.0	85.7	Claude
Average	–	**87.1**	**86.0**	–

Experiments 5-10 revealed highly competitive performance between Claude and Microsoft Copilot, with Claude averaging 87.1 and

Copilot slightly behind at 86.0. Each model showcased strengths in different scenario types, highlighting their complementary capabilities. Microsoft Copilot excelled in security-focused testing (Experiment 5), handling complex business logic (Experiment 6), and performing performance validation (Experiment 8) with technical precision and quantifiable metrics. On the other hand, Claude demonstrated superiority in integration testing (Exp. 7), comprehensive edge case coverage (Exp. 9), and full-system evaluation (Exp. 10), excelling through systematic thoroughness and strong requirements traceability. These results emphasised how the two models excelled in different testing domains, each bringing unique strengths to the table.

4.4 Comparative Analysis

Claude employs a methodical approach, analysing requirements exhaustively to identify all testable conditions and generating corresponding test cases with clear traceability. This thoroughness ensures comprehensive coverage, encompassing security, performance, reliability, and usability, as well as functional validation. While this results in large test suites that require more execution effort, Claude prioritises completeness, ensuring that no aspect of the system is overlooked. Its emphasis on systematic thoroughness makes it particularly well-suited for comprehensive testing scenarios, though it demands more resources in terms of time and processing.

Microsoft Copilot, on the other hand, focuses on generating technically precise, immediately actionable test specifications. Its test cases include specific technical details such as API endpoints, HTTP status codes, JWT token handling, database transaction expectations, and protocol specifications, all of which are highly valuable for developers and automation engineers. This precision enables easier test automation and simplifies technical troubleshooting. Copilot strikes a balance between coverage and efficiency, generating more concise test suites through intelligent scenario consolidation, making it an ideal choice for environments where technical accuracy and automation are key priorities.

4.4.1 Overall Assessment and Claude Selection

Aggregating across all ten Phase 2 experiments yields overall performance assessment.

Table 4.4: Phase 2 Overall Performance

Model	Experiments 1–4 Avg	Experiments 5–10 Avg	Overall Avg	Total Wins
Claude AI	97.3	87.1	91.2	7 of 10
Microsoft Copilot	93.4	86.0	88.9	3 of 10
ChatGPT	73.0	–	73.0	0 of 4

Claude achieved the highest overall performance in Phase 2 with a score of 91.2, showcasing exceptional results in the early experiments and maintaining competitive performance in the later stages, winning 7 of the 10 experiments. Microsoft Copilot also demonstrated strong performance, with a score of 88.9, offering particular advantages for automation-focused workflows and technically oriented testing teams due to its precision and efficiency.

Claude's selection for Phase 3 was driven by several key factors: its highest overall performance, indicating the most consistent capability across various testing scenarios, and its superior comprehensive coverage, which is essential for thorough system validation. Claude's systematic approach aligned with professional quality assurance practices, emphasising auditability and detailed requirements traceability. Furthermore, its balanced strengths across all evaluation criteria suggested fewer systematic blind spots, making it the most well-rounded model for further evaluation against human-generated test cases.

5 Comprehensive Human vs. AI Evaluation

This chapter presents the findings from Stage 3, which compare Claude, the highest-performing AI model from Stage 2, against professional human quality assurance engineers through fifteen controlled experiments. The goal of the investigation was to gather detailed empirical evidence regarding where AI excels, where it falls short, and how AI and human capabilities might complement each other in practical testing workflows. The experiments progressed from an exploratory assessment (Experiments 1-8) to controlled refinements (Experiments 9-15), systematically addressing the limitations identified in earlier phases while building a comprehensive understanding of the strengths and weaknesses of both AI and human testers.

5.1 Experimental Design

Stage 3 employed a systematic methodology to compare Claude-generated test cases with those written by human quality assurance professionals across multiple dimensions. The human test cases were developed by experienced professionals who created documentation for CORADS using the same SRS that was provided to Claude. This ensured a fair comparison, as both the AI and human test cases were based on identical requirement specifications, allowing for a direct evaluation of the models' performance in generating high-quality test cases.

5.1.1 Evaluation Framework

Stage 3 extended the evaluation beyond the Stage 2 criteria by incorporating expanded metrics that address specific concerns relevant to production testing environments:

Accuracy Measures the factual correctness of the test cases, assessing the percentage of test specifications that align with the actual SRS requirements, ensuring there is no hallucinated or misinterpreted content.

Completeness Evaluates the breadth of coverage, specifically the percentage of requirements that receive appropriate test coverage, including both functional and non-functional specifications.

Requirements Traceability Matrix (RTM) Coverage Quantifies the percentage of requirements that are explicitly linked to at least one corresponding test case through requirement identifiers, ensuring traceability from requirements to test cases.

Test Coverage Assesses the systematic attention given across various requirement categories, testing techniques, and scenario types, ensuring a comprehensive approach to testing.

Oracle Specificity (1-5 scale) Measures the precision of expected results, evaluating whether the expected outcomes are measurable and objective versus being vague or ambiguous.

Hallucination Rate Counts the number of fabricated specifications per 100 test steps, representing instances where AI generated content that was unsupported by the SRS documentation.

Duplicate Reduction Measures the percentage of redundant test cases identified and eliminated through analysis, assessing how well the model avoids unnecessary repetition in its test suite.

Risk Prioritization Alignment Uses Cohen's kappa coefficient to measure agreement between AI-generated and human-generated priority assignments, focusing on whether both approaches align in identifying critical areas for testing.

Efficiency Quantifies the time and effort required to generate the test cases, evaluating how efficiently each model can produce high-quality test specifications.

5.1.2 Dual-Perspective Verification

All Stage 3 experiments employed a dual-perspective analysis, combining the researcher's independent evaluation with LLM-assisted verification. Two independent human raters evaluated all test cases using structured rubrics to ensure consistency and objectivity. Cohen's

kappa coefficient was used to measure inter-rater reliability, with a target threshold of 0.70 indicating substantial agreement between the raters. Statistical analysis included paired t-tests and Wilcoxon signed-rank tests to assess significance at a $p < 0.05$ threshold. Additionally, Cohen's d was calculated to measure the effect size, providing insight into the magnitude of performance differences between the AI and human-generated test cases.

5.1.3 Experimental Progression

Experiments 1-8 took an exploratory approach to identify recurring patterns in Claude's strengths and limitations. Early findings highlighted several specific concerns: hallucinations, where Claude invented scenarios not supported by the SRS content, traceability gaps, despite overall strong performance, and misalignment in risk prioritisation with human business intuition.

In response, Experiments 9-15 implemented controlled refinements to address these issues specifically. Modifications included explicit grounding constraints instructing Claude to mark any assumptions extending beyond the SRS as 'out of scope'. RTM-first templates were introduced, requiring explicit requirement mapping before generating test steps. Screen/element validation was added to ensure UI references matched the actual specifications. Additional improvements included duplicate elimination passes to avoid redundancy, policy-citation rules that demand explicit source references, oracle contract schemas to enforce measurable expectations, and risk weighting models to align prioritisation with business criticality. These refinements were designed to enhance Claude's performance and address the identified limitations, ensuring more reliable and relevant test case generation.

5.2 Overall Quantitative Results

Table 6.1 presents aggregate performance across all fifteen Phase 3 experiments.

Table 5.1: Stage 3 Overall Performance Summary

Metric	Claude AI Avg	Human Avg	Δ (Claude – Human)	Interpretation
Accuracy %	96.2	92.8	+3.4	AI fewer factual errors
Completeness %	72.8	87.1	-14.3	Humans infer missing FRs
RTM Coverage %	100.0	55.0	+45.0	AI perfectly traceable
Test Coverage %	95.0	88.0	+7.0	AI more systematic
Oracle Specificity (1–5)	5.0	1.8	+3.2	AI measurable expectations
Hallucination Rate (per 100)	0.55	0.88	-0.33	Lower AI over-inference
Duplicate Reduction %	50.0	0.0	+50.0	AI identifies redundancy
Efficiency (relative)	8× faster	1×	–	Major productivity gain

Claude demonstrated superiority in several key areas, including accuracy, RTM coverage, test coverage, oracle specificity, and efficiency. However, human testers outperformed Claude in completeness, as they were able to leverage contextual inference to identify implicit requirements that Claude missed. Statistical significance testing confirmed several key differences: Claude had a significant advantage in accuracy ($p = 0.012$, Cohen's $d = 1.01$, indicating a significant positive effect), RTM coverage ($p < 0.001$, Cohen's $d = 2.3$), and efficiency ($p < 0.001$). Additionally, the inter-rater reliability between the human raters was strong, achieving a of 0.72, which surpassed the target threshold of 0.70, indicating substantial agreement. These findings highlight Claude's strengths in generating precise, efficient test cases, while also recognising that human testers excel in addressing implicit requirements and ensuring comprehensive coverage.

5.3 Detailed Analysis by Dimension

5.3.1 Accuracy and Hallucination Control

Claude achieved 96.2% accuracy, outperforming human testers at 92.8%, producing fewer factual errors and misalignments with the SRS specifications. However, early experiments revealed concerns about hallucinations, where Claude generated scenarios, data fields, or workflows not supported by the documentation. In Experiment 1,

the model generated test cases referencing "urban/rural driver types," a classification that did not exist in the CORADS SRS. Claude inferred this categorisation based on its domain knowledge of transportation systems, but it introduced unnecessary scope creep into the requirements. The hallucination rate was initially 0.92 fabricated specifications per 100 test steps.

To address this, Experiments 9-15 implemented grounding constraints, explicitly instructing Claude to mark any assumptions extending beyond the documented requirements as 'out of scope'. This intervention successfully reduced the hallucination rate to 0.55 per 100 test steps, reflecting a 40% reduction. The remaining hallucinations primarily involved reasonable inferences about error handling and edge cases, where the SRS was silent, and these deviations were less severe compared to the model's invention of functional capabilities.

Human testers exhibited a hallucination rate of 0.88, slightly higher than the refined Claude. However, humans' "over-inference" manifested differently, rather than inventing new features, they made assumptions based on their experience, such as assuming operational procedures, organisational policies, or technical implementations. While these contextual assumptions were often valuable, they occasionally led to test requirements that were not directly supported by the provided documentation.

5.3.2 Completeness and Contextual Understanding

Human testers achieved 87.1% completeness, outperforming Claude's 72.8%, demonstrating a superior ability to infer implicit requirements and contextually implied functionality. This human advantage stems from several key factors.

Professional testers often recognise standard functionality that is commonly omitted from SRS documentation, as authors may assume a universal understanding of certain features. For example, human test suites included comprehensive logout testing, session timeout validation, and concurrent login handling, despite the SRS having minimal coverage. Claude generated these scenarios inconsistently, requiring explicit specification.

Business logic interpretation also favoured human testers. When the SRS described payment processing without specifying failure

handling, recovery procedures, or reconciliation workflows, human testers proactively generated comprehensive failure scenarios based on standard financial transaction practices. Claude, while explicitly addressing stated error conditions, was less proactive in exploring unstated but practically necessary failure modes.

Domain knowledge further enabled humans to identify missing but critical test coverage. For example, human test suites included tests for regulatory compliance, accessibility requirements, and production operational concerns, all reflecting professional experience in real-world testing scenarios. Claude demonstrated systematic coverage of the stated requirements, but it lacked the initiative to identify omissions in specification completeness, which is an essential aspect of thorough testing.

5.3.3 Requirements Traceability

Claude achieved a perfect 100% RTM coverage, with explicit requirement identifiers for every test case, ensuring that each test was clearly linked to a corresponding requirement. In contrast, human test suites reached only 55% RTM coverage, with 25.4% of test cases lacking any explicit requirement traceability.
Experiment 10 specifically evaluated traceability through the **RTM-First generation approach**. Claude produced comprehensive mapping, generating **96 test cases** that covered all **87 functional requirements** with explicit `FR_ID` tags, averaging 1.1 tests per requirement. In comparison, the human suite included 130 test cases but only covered 48 of the 87 requirements (55.2%), with 33 test cases representing integration tests that lacked explicit requirement mapping, requiring inference to establish traceability.

The missing human coverage highlighted significant gaps in critical functionality, such as security (e.g., OTP, payment gateways, **vouchers), advanced features (e.g., role management, performance reports, heat maps), tracking/analytics capabilities, and communication systems. These gaps represent a primary audit readiness concern, particularly in regulated environments that require comprehensive evidence of traceability.

Claude's systematic approach ensures that every requirement receives thorough testing attention while maintaining clear audit trails.

In contrast, the human tendency toward pragmatic, execution-oriented testing sometimes sacrifices documentation rigour in favour of perceived practical efficiency, potentially compromising traceability in environments that demand high accountability.

5.3.4 Oracle Quality and Measurability

Oracle specificity revealed dramatic differences between Claude and human-generated test cases. Claude consistently achieved a perfect 5.0 rating, generating measurable and objective expected results. In contrast, human test cases averaged 1.8/5.0, with 19% exhibiting vague, unverifiable expectations. Experiment 14 focused explicitly on oracle quality by enforcing the Oracle Contract schema. Claude-generated test cases specified clear, measurable expectations, such as:

- Observable behaviours (e.g., UI state changes, system responses, database modifications),
- Precise thresholds (e.g., response time < 2.0 seconds, success rate $\geq 99.5\%$),
- Specific system codes (e.g., HTTP 201 Created, HTTP 401 Unauthorised),
- Data side-effects (e.g., `table.field` = `expected_value` with concrete values).

On the other hand, human test cases often use vague expectations, such as "system works correctly," "appropriate error message displayed," or "data saved successfully." These general formulations lack clarity and provide insufficient guidance for pass/fail determination, complicating both manual test execution and automation implementation.

This difference in oracle quality directly impacts test reliability and automation readiness. Claude-generated tests enable objective and repeatable validation, ensuring consistent results across all tests. In contrast, human-generated tests sometimes require tester interpretation for execution, introducing subjectivity and reducing repeatability, a significant challenge for effective test automation.

5.3.5 Risk Prioritization and Business Judgment

Risk prioritisation emerged as Claude's most significant limitation, with a Cohen's kappa of 0.34, indicating only fair agreement between the AI's and humans' priority assignments, revealing a systematic misalignment. Experiment 15 examined the patterns in priority assignment. Claude employed a logical, but context-insensitive approach. For example, it assigned High priority to functional correctness and data integrity concerns, Medium priority to performance requirements unless explicit SLA specifications were given, and Low to Medium priority to UI/UX issues. While this approach was systematic, it lacked the necessary business context.

In contrast, human testers applied experienced judgment, considering multiple factors such as revenue impact (e.g., payment processing received the highest priority regardless of technical complexity), regulatory requirements (e.g., security and compliance marked as critical), user base scale (e.g., features affecting all users prioritized over administrative functions), and historical defect patterns (e.g., areas with past production issues receiving heightened attention).

Specific misalignments included customising the Claude marking profile as a Medium priority. In contrast, humans marked it as Low, while Claude treated all OTP scenarios as equally High priority. In contrast, humans distinguished between critical-path authentication (High) and optional verification (Medium). Claude assigned a uniform Medium priority to all reporting features, while humans prioritised revenue-impact reports as High and informational dashboards as Low. This limitation stems from Claude's lack of organisational context, business strategy understanding, and historical operational knowledge—critical elements that senior QA professionals bring to the table with their deep domain expertise.

5.3.6 Test Suite Structure and Redundancy

Claude demonstrated superior duplicate identification and elimination in its test suite. Initially, 50% of the test cases generated contained redundancy, as they tested functionally identical conditions with only superficial variations. Upon review, Claude successfully identified

and consolidated these redundant test cases, improving the overall efficiency and clarity of the suite.

In contrast, human test suites exhibited minimal redundancy (0% measured). However, this result was partially due to a conservative approach to test generation, where human testers often prioritised fewer, more focused test cases. While this approach helped avoid redundancy, it sometimes resulted in undercoverage of specific requirements. In essence, human testers achieved efficiency by reducing the number of test cases, but this occasionally resulted in less comprehensive coverage rather than the intelligent consolidation of redundant tests. Thus, while Claude excelled at identifying and eliminating redundancy to produce more comprehensive test suites, human testers achieved efficiency by generating fewer test cases, sometimes at the cost of full coverage.

5.3.7 Efficiency and Productivity

The efficiency differences between Claude and human testers were dramatic. Claude generated comprehensive test suites in approximately 6 minutes per module, while human testers required an average of 65 minutes per module, representing a 10× productivity difference. This efficiency advantage becomes even more pronounced in large projects. For the complete CORADS system, Claude generated the whole test suite in under one hour, whereas human development required multiple days of professional QA engineer effort. However, comparing raw generation speed alone oversimplifies the value assessment. Human time investment involved not only the mechanical task of documenting test cases but also contextual analysis, business logic reasoning, and the application of domain expertise, cognitive tasks that are often not directly measurable in terms of time spent. These aspects of the process, which involve interpreting requirements, understanding the broader context, and making informed judgments about test priorities, are partially orthogonal to the mechanical work of generating test cases, making it a more complex and nuanced contribution to the testing process.

5.4 Qualitative Observations and Patterns

Beyond quantitative metrics, Phase 3 revealed qualitative patterns characterizing AI versus human testing approaches:

The analysis of test case generation reveals distinct, yet complementary, strengths and weaknesses for the AI model (Claude) and human testers.

Claude's Strengths

Systematic Comprehensiveness Claude demonstrated notable strengths in systematic comprehensiveness, ensuring that no stated requirement went unaddressed.

Structural Consistency Provided uniform quality across all test cases.

Documentation Completeness Adhered to the full IEEE 829 standard, including all essential components.

Technical Precision Specified exact validation criteria for test cases.

Traceability Discipline Was exemplary, maintaining perfect requirement mapping.

Objectivity Claude's approach was free from bias, ensuring objective coverage that was not influenced by tester preferences or time pressures.

Claude's Weaknesses

Contextual Blindness Prevented it from inferring implicit requirements that weren't explicitly stated.

Business Logic Naivety Hindered its understanding of domain-specific workflows, leading to missed considerations in areas where industry knowledge would be beneficial.

Risk Insensitivity Claude's priority assignments lacked the necessary business context, resulting in priorities that did not reflect true risk or impact.

Environmental Unawareness Platform-specific, deployment-specific, or operational concerns were often omitted.

Over-Literal Interpretation Generated tests only for the explicitly stated scenarios, missing potential edge cases or unspoken expectations.

Human Strengths

Contextual Reasoning Human testers excelled in contextual reasoning, using their ability to fill gaps in the SRS with reasonable assumptions where information was missing.

Business Judgment Allowed them to prioritize tests based on an understanding of potential impact.

Domain Expertise Helped identify standard functionality that was often omitted from the documentation.

Risk Awareness Drove human testers to focus on critical areas, ensuring the most important aspects of the system were thoroughly tested.

Execution Pragmatism Human testers balanced the need for thoroughness with the realities of available resources.

Environmental Sensitivity Enabled human testers to address platform-specific and operational concerns.

Human Weaknesses

Inconsistency Often led to quality variation across different testers and time periods, which could affect the overall reliability of test cases.

Documentation Gaps Were common, leaving traceability incomplete, making it harder to link test cases directly back to the original requirements.

Bias Sometimes influenced coverage, with testers focusing more on familiar or preferred areas, potentially overlooking other important requirements.

Efficiency Limitations Constrained the thoroughness of human-generated test cases.

Oracle Vagueness Reduced the repeatability of tests and hindered automation readiness, as many human tests lacked the precise, measurable criteria required for consistent automation.

In summary, Claude demonstrated strengths in structure, consistency, and traceability, while humans excelled in contextual reasoning, business judgment, and domain-specific expertise. However, both also had distinct weaknesses: Claude's limitations in context and business logic understanding, and humans' challenges with inconsistency, efficiency, and documentation rigor. These strengths and weaknesses highlight the **complementary nature** of AI and human capabilities in a testing environment.

Hybrid Test Generation Workflow

Analysis suggests neither a pure AI nor a pure human approach achieves optimal results. Evidence supports a **hybrid workflow** combining complementary strengths:

1. **Phase A: AI-Driven Draft Generation**
    - Claude generates a comprehensive baseline test suite with perfect traceability, measurable oracles, and systematic coverage of stated requirements.
    - Deliverables include complete test cases with IEEE 829 structure, a requirements traceability matrix (RTM), and a coverage dashboard identifying tested and untested requirements.
2. **Phase B: Human Expert Validation and Enhancement**

- Professional QA engineers review the AI-generated suite, applying contextual knowledge to:
  - Identify missing implicit requirements.
  - Adjust risk priorities based on business impact.
  - Add environment-specific scenarios reflecting deployment realities.
  - Refine test data selections based on production patterns.
  - Validate business logic interpretation accuracy.

3. **Phase C: AI-Assisted Analysis and Maintenance**
   - Claude performs consistency checking across the enhanced suite.
   - Identifies remaining gaps through RTM analysis.
   - Suggests additional scenarios for incomplete coverage areas.
   - Maintains traceability as requirements evolve.
   - Generates regression test selection recommendations.

This hybrid approach achieved 96% accuracy, 95% coverage, $< 1\%$ hallucination rate, and an $8\times$ **efficiency improvement** compared to purely human workflows while preserving human judgment advantages.

Statistical Validation

Comprehensive statistical analysis validated Phase 3 findings:

Primary Outcome (Accuracy) 95% confidence interval for Claude accuracy $= 96.2 \pm 2.8\%$. Significance testing versus human baseline yielded $p = 0.012$ ($p < 0.05$), confirming a statistically significant advantage. The effect size Cohen's $d = 1.01$ represents a large positive effect, indicating not merely statistically significant but practically meaningful improvement.

Inter-Rater Reliability Cohen's $\kappa = 0.72$ exceeds the target 0.70 threshold, demonstrating substantial inter-rater agreement and validating the evaluation methodology reliability.

Secondary Outcomes

- RTM coverage advantage achieved $p < 0.001$ with Cohen's $d = 2.3$ (very large effect).
- Oracle specificity difference reached $p < 0.001$ with Cohen's $d = 1.8$ (large effect).
- Completeness disadvantage measured $p = 0.03$ with Cohen's $d = 0.6$ (medium effect).

Statistical rigor establishes Phase 3 findings meet academic and industrial credibility thresholds for reproducible scientific conclusions.

6 Challenges and Limitations

This chapter explores the challenges encountered during the research and highlights the inherent limitations of both AI-based test generation and the research methodology itself. Understanding these constraints is crucial for accurately interpreting the findings, recognising the limitations of generalizability, and identifying potential directions for future investigation. The analysis distinguishes between AI-specific technical limitations, methodological research constraints, and practical implementation challenges that organisations may face when deploying such systems. By addressing these factors, the chapter provides a comprehensive view of the study's scope and applicability, while offering insights into areas that may require further exploration or refinement in the context of real-world adoption.

6.1 AI-Specific Technical Challenges

Large language model test generation exhibits characteristic limitations stemming from fundamental AI capabilities and training approaches. These technical challenges manifested consistently across experiments despite prompt engineering efforts and controlled refinements.

6.1.1 Hallucination and Over-Inference

Hallucination, or the generation of content unsupported by source documentation, represents one of the most critical limitations of AI-based test generation. In the early Phase 3 experiments, Claude demonstrated an initial hallucination rate of 0.92 fabricated specifications per 100 test steps. While controlled refinements, such as the introduction of grounding constraints, reduced this rate to 0.55, the complete elimination of hallucinations proved unattainable.

The manifestations of hallucination varied in severity. Minor cases involved reasonable assumptions about standard functionality. For example, Claude generated test cases for logout confirmation dialogues or session timeout handling, even though the SRS did not explicitly mention these standard features. While technically hallucinations,

these inferences often proved valuable for test completeness, as they addressed standard functionality that would typically be expected in the system; however, more problematic cases involved entirely fictional requirements.

Experiment 1 revealed that Claude generated an "urban/rural driver classification" along with associated differential pricing and route optimisation logic, which were completely absent from the CORADS SRS. These fabrications pose a risk of scope creep, development misalignment, and testing functionality that does not exist, leading to wasted resources and potential confusion during validation.

The challenge of hallucination becomes more pronounced when the quality of documentation varies. When requirements are well-specified, hallucinations are less likely to occur because the models can find explicit guidance. However, ambiguous or incomplete specifications often trigger the model's inference mechanisms, increasing the likelihood of hallucination. This creates a paradox: AI assistance seems most valuable in scenarios with poor documentation, yet it performs most reliably when the specifications are comprehensive.

Experiment 13's resilience testing highlighted systematic over-inference. Claude generated 28 fault-tolerance test scenarios, of which only 2 (7%) were explicitly supported by the SRS requirements. The remaining 24 scenarios represented industry best practices such as retry mechanisms, exponential backoff, failover strategies, and caching implementations, all inferred from domain knowledge despite the SRS's silence on these areas. While these tests could identify valuable system improvements, they were hypothetical requirements, which could confuse development priorities and misdirect testing efforts toward non-critical functionality.

6.1.2 Contextual Understanding Limitations

AI models demonstrate superficial rather than deep contextual understanding. This limitation manifests in several ways affecting test generation quality.

Implicit Requirement Recognition: Human testers excelled at identifying unstated but necessary functionality based on domain expertise and an understanding of common practices. For instance, standard features such as comprehensive logout workflows, concur-

rent login handling, and session management were tested by humans, despite minimal coverage in the SRS. Claude, on the other hand, generated these test cases inconsistently, requiring explicit specification to ensure reliable coverage of these features.

Business Logic Interpretation: When the SRS described payment processing but lacked detailed failure handling specifications, human testers drew on their knowledge of the financial transaction domain to generate comprehensive failure scenarios. Claude addressed the explicitly stated error conditions, but it lacked the initiative to explore unstated, yet practically necessary failure modes. This difference underscores humans' ability to recognise business-critical scenarios even when documentation is inadequate. At the same time, Claude's reliance on explicitly defined requirements limited its ability to infer critical scenarios beyond the SRS.

End-to-End Flow Understanding: In Experiment 2, Claude demonstrated difficulties with sequencing dependencies in complex workflows. It occasionally mis-sequenced test steps in multi-stage processes, generating test cases that attempted operations before the prerequisite conditions were established. Human testers, however, naturally recognised logical flow requirements and temporal dependencies, even when the specifications presented requirements in non-sequential order. This ability to understand and correctly sequence end-to-end flows highlighted the advantage of human judgment in navigating complex workflows.

Integration Testing Gaps: Experiment 9 revealed a significant gap in Claude's testing suite, as it omitted integration testing entirely, despite the CORADS system comprising multiple interconnected modules. In contrast, the human test suite included 33 integration tests, covering Firebase communication, server interactions, network handling, local database synchronisation, and push notification delivery. Claude's focus on individual requirement validation prevented it from recognising the critical need for inter-module validation, which is essential for testing how different components of the system work together.

6.1.3 Risk Prioritization Deficiencies

Risk-based testing prioritization proved to be Claude's most significant limitation, with only fair agreement ($= 0.34$) between its priority assignments and those made by human professional judgment. Claude relied on a systematic, but context-insensitive algorithm to assign priorities: functional correctness and data integrity concerns were marked as High, performance requirements were given Medium priority unless explicit SLAs were provided, and UI/UX issues were marked as Low to Medium. While this approach was logical and consistent, it lacked an understanding of the business context, leading to misalignments with the real-world testing needs that human testers are better equipped to handle.

In contrast, human testers applied their experienced judgment, considering factors such as revenue impact, regulatory requirements, user base scale, and historical defect patterns. For example, payment processing was prioritised as high, not because it was technically more complex than other features, but because failures in this area directly impact revenue. Security and compliance testing were also given critical priority due to regulatory exposure, and features impacting all users were prioritised over administrative functions, regardless of their technical complexity. Additionally, human testers considered organisational memory when prioritising areas with a history of production issues.

There were several misalignments between Claude's and the human testers' priorities, such as profile customisation, which Claude marked as Medium. In contrast, humans marked it Low, and optional OTP verification, which Claude treated as High across the board, while humans differentiated it based on authentication criticality. Reporting features were another example, with Claude treating all as Medium, while humans gave revenue-impact reports a High priority and informational dashboards a Low priority. These discrepancies stem from Claude's lack of access to organisational context, business strategy, and historical operational data key insights that senior QA professionals bring to testing. These professionals leverage domain expertise to prioritise testing based on real-world impact, something that Claude, as an AI, currently cannot replicate.

6.1.4 Environmental and Operational Unawareness

AI-generated tests had a limited understanding of real-world factors, such as deployment environments and operational challenges. For example, human testers would consider factors such as ensuring an app works across different Android versions (7-10), testing its behaviour on various browsers like Chrome, Opera, Edge, and Firefox, or evaluating how the app performs under different mobile network conditions and GPS accuracy issues.

Claude was great at generating technically accurate functional tests, but he often missed testing for environmental variations unless specifically instructed to do so. It also didn't focus much on production-related concerns, like verifying that the system deploys correctly, integrates with monitoring tools, supports log analysis, or handles rollback scenarios in case of failure. Human testers, on the other hand, naturally incorporated these aspects into their tests due to their real-world experience with deploying and supporting systems in live environments.

6.2 Methodological Limitations

The research methodology, while systematic and rigorous, exhibits inherent limitations affecting findings generalizability and interpretation.

6.2.1 Single SRS Document Constraint

The investigation utilised a single SRS document, CORADS, as the primary experimental artefact. This decision, while practical for enabling controlled comparisons, limits the generalizability of the findings. CORADS represents a specific domain (location-based advertising), a medium-scale web and mobile platform, and a professional IEEE-compliant specification with a certain level of documentation quality and requirement characteristics.

In different domains, AI performance might vary significantly. For example, safety-critical systems, such as medical devices, aviation, or automotive applications, require much more exhaustive validation, with different testing philosophies than those used in commercial web

applications. Similarly, highly regulated domains such as financial services, healthcare, or government require specific compliance testing and audit trails, which may highlight different strengths of AI. Embedded systems, real-time systems, and algorithm-intensive applications also present unique testing challenges that could influence how AI performs in these contexts.

The quality of the specification also plays a crucial role in AI performance. This research employed a professionally written, well-structured SRS that adhered to IEEE standards, enabling AI to perform at its best. However, organisations with informal requirements documentation, such as user-story-based specifications or poorly maintained requirements, may see different levels of effectiveness from AI tools. On the other hand, organisations with even more rigorous specification practices might see improved AI performance, as the more precise and more detailed the specifications, the better AI can generate relevant and accurate test cases.

6.2.2 Model Version Temporality

The research evaluated specific versions of AI models available during the investigation period (late 2024 to early 2025). Given that large language models evolve rapidly through continuous training, architectural improvements, and specialised fine-tuning, the findings represent performance snapshots that may differ from earlier versions or future releases.

This temporality influences both absolute performance assessments and the relative model rankings observed during the study. While Claude demonstrated superiority during the evaluation, models like Microsoft Copilot, ChatGPT, or others could produce different results with updated versions. Additionally, entirely new model families might emerge, exhibiting different capability profiles. Therefore, the findings should be viewed as a reflection of the current state-of-the-art in AI, rather than definitive, permanent limitations of AI technology.

6.2.3 Evaluation Subjectivity

Despite the use of dual-perspective analysis, structured rubrics, and statistical validation, the evaluation still involved some degree of subjective judgment. Assessing test case quality requires interpretation, particularly when considering factors like appropriateness, practical utility, and alignment with professional standards. Different evaluators prioritise various quality dimensions or apply different thresholds for what they deem acceptable.

Inter-rater reliability ($= 0.72$) indicates substantial agreement between evaluators, but it also suggests that perfect consistency was not achieved. The 28% unexplained variance highlights the legitimate subjectivity involved in quality assessment. While the evaluation rubrics provided helpful guidance, they could not eliminate judgment calls, especially in cases where the distinction between borderline and non-borderline situations was unclear, or when making trade-off decisions or determining the relative importance of specific criteria. This inherent subjectivity reflects the complex nature of test case evaluation, where different perspectives and priorities can influence the final assessment.

6.2.4 Training Data Quality and Adversarial Risks

The research identified two interconnected concerns regarding the foundation of generative AI model capabilities: training data quality and susceptibility to data manipulation.

Public Data Quality Constraints

Large language models (LLMs) are primarily trained on publicly available text, resulting in significant quality limitations in technical and domain-specific contexts. This is particularly evident in areas like software requirements engineering and test case generation, where the gap between the training data and real-world professional standards becomes apparent. The majority of publicly available Software Requirements Specification (SRS) documents come from:

- Academic course projects and student assignments

- Open-source project documentation, which is often informal or incomplete
- Small organizations with limited resources for quality assurance
- Tutorials and educational materials that prioritize teaching over professional rigor

In contrast, high-quality SRS documents from large enterprises (e.g., Fortune 500 companies, government contractors, or developers of safety-critical systems) remain proprietary and inaccessible for model training. These documents typically include:

- Detailed database schemas and data dictionary specifications
- Comprehensive API contracts with request/response examples
- System architecture diagrams with deployment details
- Non-functional requirements with measurable SLAs
- Regulatory compliance mappings and audit trail specifications

Since these high-quality documents are not part of the training data, the models are unable to replicate the level of technical detail, structural rigor, and domain-specific terminology that characterizes professional software engineering. This gap directly limits the models' ability to generate professional-grade test artifacts.

Manipulation and Adversarial Risks

The reliance on publicly available data introduces another risk: intentional manipulation. Since these models learn from any accessible text, there's a possibility that malicious actors could inject misleading or harmful information into public data sources, which could influence the model's behaviour. Examples observed in this research include:

- **Outdated Best Practices:** The models occasionally recommended deprecated testing approaches or obsolete security practices, likely learned from older documentation that remains publicly accessible despite being outdated.

- **Framework-Specific Biases:** Some models appeared overly familiar with popular open-source frameworks, but showed a lack of understanding of enterprise-level or proprietary technologies, which reflects the open-source-heavy nature of their training data.

- **Terminology Inconsistencies:** Conflicting or inconsistent technical terms appeared in the models' outputs, suggesting that the models were trained on diverse data sources that did not align on terminology.

Although there was no direct evidence of intentional data poisoning, the potential risk is significant. A skilled attacker could manipulate public repositories with subtle misinformation, causing the AI to generate insecure test cases or overlook critical vulnerabilities.

Implications for Research Validity and Practical Deployment

The limitations of the training data impact the generalizability of the research findings. The 96.2% accuracy and high traceability achieved by Claude AI were based on the publicly available CORADS SRS document, which may differ significantly from proprietary enterprise-level specifications. As such, organisations with mature software engineering practices and higher-quality documentation might see different performance results when applying AI models to their own systems.

For practical deployment, these data quality limitations suggest several strategic responses:

- **Custom Fine-Tuning:** Organizations should consider developing domain-specific models that are fine-tuned using their own SRS documents, API documentation, and historical test suites. This approach will ensure the models are better aligned with the organization's specific needs and standards (as discussed in Section 8.4.1).

- **Human Oversight:** Given the concerns over the quality of training data, human validation of AI-generated artefacts is crucial. This isn't just recommended it's essential to ensure accuracy, contextual appropriateness, and alignment with business priorities.

- **Controlled Data Sources:** Organisations should select AI models trained on curated data sources with clear provenance, rather than models trained on unrestricted, web-scale data. This would mitigate the risk of adversarial contamination and ensure more reliable performance in real-world contexts.

6.2.5 Human Baseline Variability

Human test case quality can vary significantly depending on factors such as individual tester experience, organisational context, time pressures, and the project phase. In this research, experienced QA professionals were used to produce high-quality human baselines, ensuring a reliable comparison with AI-generated test cases. However, the performance of junior testers, teams working under time constraints, or organizations with less rigorous processes might lead to different results, which could influence the AI-vs-human comparison and potentially alter the conclusions drawn.

The human test cases used in this study were developed within a real project context, which included time constraints, tool limitations, and other practical considerations. While idealised comparisons, assuming unlimited time and resources, might paint a picture of "perfect" human performance, these would lack the ecological validity of real-world conditions. Such idealised comparisons would focus on theoretical purity, but they would not accurately reflect the practical realities faced by testers in real projects.

6.3 Practical Implementation Challenges

Beyond technical and methodological limitations, organizations adopting AI test generation face practical implementation challenges affecting deployment success.

6.3.1 Workflow Integration Complexity

Integrating AI test generation into existing quality assurance workflows requires significant adaptation to traditional processes. Typically, these workflows assume that test documentation is authored by humans, with well-established review, approval, and maintenance pro-

cedures in place. AI-generated content, however, demands a different approach to handling and processing.

One key challenge is establishing appropriate human oversight mechanisms. Pure AI generation without human validation risks introducing hallucinated or contextually inappropriate tests, which could lead to faulty testing outcomes. On the other hand, excessive review requirements could undermine the efficiency gains that make AI adoption attractive in the first place. Therefore, finding the right balance between human-AI collaboration requires ongoing experimentation and organisational learning to determine the most effective workflows.

Another challenge is integrating AI with existing test management tools, which were traditionally designed for human users to create and update test cases through interactive interfaces. AI-driven test case generation necessitates API integration, format standardisation, and metadata mapping to enable programmatic test case injection. For organisations using legacy test management systems that lack these capabilities, this presents a significant barrier to adoption.

Moreover, version control and change management processes require reconsideration. When AI regenerates test suites based on updated requirements, it becomes challenging to distinguish meaningful changes from spurious variations through traditional diff-based reviews. Establishing the right granularity, change tracking, and approval workflows is critical for managing test case updates effectively and efficiently. This requires careful process design to ensure that both the AI and human efforts are aligned and properly managed.

6.3.2 Team Adoption and Cultural Factors

Resistance from QA professionals poses a significant barrier to the adoption of AI-based testing tools. Concerns about job security, skill obsolescence, and professional identity often deter individuals from embracing these technologies. To overcome these concerns, organisations must address them with transparent communication, reskilling opportunities, and a clear vision for the evolution of roles in the face of AI integration.

When AI tools are successfully adopted, they can reposition testers as higher-value contributors in the testing process. Instead of focusing solely on test case creation, testers can take on roles such as require-

ments quality validators, contextual gap identifiers, risk prioritisation experts, exploratory testing specialists, and test suite architects. However, this evolution requires new skill development in areas like AI tool management, prompt engineering, output validation, and effective human-AI collaboration. Organisations must invest in training and support to equip their teams with the necessary skills to thrive in this changing landscape.

Building trust in AI-generated tests requires demonstrated reliability and accuracy. Initial AI-generated tests that contain hallucinations or contextual misunderstandings can undermine confidence in the tool's effectiveness. To establish trust, a phased rollout is essential, starting with low-risk areas where AI testing can be introduced and refined. Continuous monitoring for quality issues and transparent acknowledgement of limitations help ensure that trust is built gradually, enabling testers to become more comfortable with the AI tools over time.

6.3.3 Cost-Benefit Ambiguity

While AI test generation offers dramatic efficiency gains (an 8× observed speedup), total cost of ownership analysis proves to be complex. AI tool subscriptions, API usage costs, infrastructure expenses, integration development, training investments, and ongoing maintenance must be weighed against the productivity improvements, quality enhancements, and competitive advantages they provide.

Hidden costs include prompt engineering effort required for high-quality outputs, human validation labour for ensuring appropriate test coverage, rework addressing hallucinated or inappropriate tests, and organisational learning curve during adoption. Break-even analysis requires a realistic assessment of all factors.

Benefits extend beyond direct time savings. Comprehensive traceability improves audit readiness for regulated environments. Measurable test oracles facilitate automation. Consistent test structure reduces maintenance overhead. Systematic coverage reduces oversight-driven defects. Quantifying these benefits challenges traditional ROI models.

6.3.4 Organizational Readiness Prerequisites

Successful AI test generation deployment requires organisational prerequisites often underestimated during adoption planning. High-quality requirement documentation proves essential, as AI performance degrades substantially with poor specifications, creating a bootstrapping problem where AI assistance appears most valuable precisely where its prerequisites are absent.

Established testing processes provide the necessary foundation. Organisations lacking systematic test case management, requirements traceability discipline, or structured quality gates face compounded challenges adopting AI tools. Addressing foundational process maturity must precede or accompany the adoption of AI.

Technical infrastructure requirements include API access to AI services (raising security and data privacy considerations for sensitive projects), integration with existing development and test management tools, sufficient computing resources for local model deployment if cloud services prove inappropriate, and robust network connectivity for cloud-based solutions.

6.4 Broader Implications and Concerns

Beyond immediate technical and practical limitations, AI test generation raises broader implications warranting careful consideration.

6.4.1 Professional Skill Evolution

The rise of AI-augmented testing shifts the skillset required for QA professionals. Traditional skills, such as manual test case authoring, are becoming less critical, while new competencies are emerging in areas like AI output validation, contextual gap identification, prompt engineering, and human-AI workflow design. As a result, educational programs and professional certifications must evolve to prepare testers for these changing demands, ensuring they are equipped with the necessary skills to collaborate effectively with AI tools.

This shift also creates potential disruption for junior testers and their career paths. Traditionally, testers would progress from manual test execution to test case authoring and eventually to test strategy

development. However, with AI handling routine test generation, this traditional career progression may be compressed. As AI assumes more of the routine and repetitive tasks, organisations will need to rethink their professional development pathways to ensure testers have clear opportunities for skill progression. This could include focusing more on higher-level strategic roles and providing growth opportunities in areas like AI tool management, test suite architecture, and risk-based testing.

6.4.2 Quality Assurance Philosophy

Relying too much on AI for testing could shift the focus from "testing to ensure quality" to simply "validating AI-generated tests." This shift, though subtle, could cause teams to stop critically thinking about what should be tested and instead focus on automatically validating whatever tests the AI suggests.

In particular, exploratory testing, which relies on a tester's intuition and creativity to uncover issues, might become less important if organisations focus too heavily on executing formal, structured test cases generated by AI. It's essential to maintain a balance between the systematic coverage that AI excels at and the creative exploration that humans bring to the testing process. This balance requires intentional effort to ensure both approaches are valued.

6.4.3 Accountability and Liability

When AI-generated tests miss critical defects leading to production failures, accountability assignment proves murky. Is fault attributed to QA team for insufficient validation, AI tool provider for capability limitations, requirements authors for specification gaps, or organizational processes for inadequate human oversight? Clear accountability frameworks must accompany AI adoption.

7 Conclusion

This thesis explored the capabilities of generative AI for automated test case generation through a systematic empirical evaluation. Five leading large language models, ChatGPT, Microsoft Copilot, DeepSeek, Grok, and Claude, were progressively assessed through requirements extraction analysis, test generation comparison, and a comprehensive evaluation against professional human testers. The research provides an evidence-based understanding of where contemporary AI excels, its limitations, and offers insights into how organisations can effectively integrate AI into their quality assurance workflows.

7.1 Research Summary

This thesis investigated the capabilities of five large language models in software testing through a comprehensive two-phase experimental framework comprising 32 controlled experiments.

Phase 1: Requirements Extraction evaluated each model's ability to extract functional and non-functional requirements from various documentation formats. Results identified ChatGPT, Microsoft Copilot, and Claude as the strongest performers, with Claude demonstrating exceptional coverage (97.5%) and traceability (97.5%).

Phase 2: Test Case Generation employed three progressive stages:

- Stage 1 conducted exploratory five-model assessment
- Stage 2 performed systematic three-model comparison
- Stage 3 executed comprehensive Claude vs human evaluation

Final results demonstrate that Claude achieves 96.2% accuracy, surpassing other models in test case generation. These results highlight Claude's strength in understanding and interpreting requirements, its ability to generate test cases with high coverage, and its efficiency in producing consistent results.

The comprehensive human comparison in stage 3 provided valuable insights, confirming that Claude's performance, while impressive, still requires human validation for certain complex scenarios, such as business logic interpretation and risk prioritization.

All experimental data, detailed results, prompts, and analysis artefacts are publicly available in the research repository, ensuring reproducibility and enabling future research to build on this foundational work.

7.2 Answers to Research Questions

The investigation addressed five research questions formulated at thesis inception. This section provides evidence-based answers synthesizing findings across all research phases.

Research Question 1: How accurately can generative AI models interpret an SRS document to produce valid test cases and a test management plan?

Contemporary state-of-the-art models, such as Claude, demonstrate high accuracy in interpreting SRS documents for test generation. Claude achieved 96.2% accuracy in the Phase 3 evaluation, generating test cases that closely aligned with actual requirements, with fewer factual errors than human-generated tests (92.8%). This superior performance can be attributed to Claude's systematic requirement processing, precise adherence to stated specifications, and consistent application of testing standards.

However, accuracy is multifaceted. While factual correctness excels, limitations in contextual interpretation become apparent, particularly in terms of completeness (72.8% versus human 87.1%). AI models excel at validating explicitly stated requirements but struggle with inferring implicit functionality, interpreting ambiguous business logic, and recognising domain-standard practices. Despite efforts to mitigate instances of hallucinations generating unsupported content, this remains an ongoing concern (0.55 incidents per 100 test steps after controlled refinements).

One of AI's strengths lies in its ability to trace requirements. Claude achieved a perfect 100% RTM coverage with explicit requirement identifiers for every test case, far surpassing human performance (55%). This traceability is invaluable for regulated environments that demand thorough audit trails.

Research Question 2: What are the differences in efficiency (time and effort) between AI-generated and human-written test cases?

The efficiency differences are striking. Claude generated comprehensive test suites in approximately 6 minutes per module, compared to 65 minutes on average for human testers, resulting in a 10× productivity advantage. For the complete CORADS system, AI generated the entire test suite in under one hour, whereas human testers would take multiple days to achieve the same result.

However, comparing raw generation speed oversimplifies the value of AI adoption. Human testers also invest time in contextual analysis, business logic reasoning, and applying domain expertise tasks that are often not related to the mechanical aspects of test case documentation. AI-generated tests require human review to ensure contextual appropriateness, check business logic accuracy, and detect hallucinations. After accounting for this validation effort, organisations adopting AI for test generation can expect a 3-5× net productivity improvement, not the 10× speedup sometimes expected.

Additional efficiency benefits arise from AI's ability to enhance traceability, thereby reducing audit preparation effort. The use of measurable test oracles facilitates test automation, and a consistent test structure reduces maintenance overhead. Moreover, AI's systematic coverage minimises the risk of defects slipping through the cracks due to oversight.

Research Question 3: How do AI-generated test cases compare to human-written ones in terms of test coverage and bug detection? Test coverage reveals complementary strengths. AI-generated tests achieved 95% coverage of stated requirements, compared to 88% for human-generated tests, demonstrating superior attention to explicitly specified functionality. Additionally, Claude'sClaude's perfect 100% RTM coverage significantly outperformed human tests, which achieved only 55%. This ensures that no stated requirement is overlooked, making AI-generated tests highly valuable in providing comprehensive testing.

However, coverage is multidimensional. Human tests demonstrated 87.1% completeness compared to Claude's 72.8%, reflecting the superior ability of human testers to identify implicit requirements, unstated functionality, and domain-standard features that were not fully captured in the SRS. Integration testing was a particular gap for AI. Human testers included 33 integration tests to validate communica-

tion between system components, while Claude omitted this crucial aspect, despite CORADS comprising interconnected modules.

In terms of bug detection, AI excels at systematic functional validation, testing non-functional requirements (98% coverage versus human 40%), exploring security scenarios, and identifying boundary conditions. On the other hand, human testers are better equipped to handle environmental variation testing (e.g., platform-specific issues, network conditions, GPS accuracy degradation), production operational concerns, and real-world failure modes, leveraging experience to identify edge cases beyond the SRS.

Oracle quality further influences bug detection effectiveness. AI-generated tests used measurable, objective expected results (5.0/5.0 specificity), enabling reliable pass/fail determination. In contrast, human tests averaged 1.8/5.0, with 19% exhibiting vague expectations, such as " system works correctly" or "appropriate error displayed," which reduced test reliability.

Research Question 4: What are the key challenges and limitations of using generative AI for test case generation? The key technical challenges include persistent hallucinations, limitations in contextual understanding, risk prioritisation deficiencies (fair agreement with human judgment at $= 0.34$), and environmental unawareness that omits platform-specific and operational considerations. Hallucinations vary in severity, from reasonable inferences about standard functionality to problematic inventions of entirely fictional requirements. In Experiment 13, Claude generated 28 resilience test scenarios, but only 7% were explicitly supported by the SRS, with the rest representing industry best practices inferred from domain knowledge. Despite refinements that reduced the hallucination rate by 40Risk prioritisation represents the most significant limitation. Claude employed systematic but context-insensitive rules, lacking the business context understanding that human professionals bring. Human testers consider factors such as revenue impact, regulatory requirements, user base scale, and historical defect patterns, which are critical for prioritising tests based on actual business needs.

Practical implementation challenges include workflow integration complexity, team adoption resistance due to concerns over job security and professional identity, and the cost-benefit ambiguity that necessitates a comprehensive total cost of ownership analysis. Many organi-

sations, especially those in need of AI assistance, may face readiness issues when adopting AI tools.

Methodological limitations include the focus on a single SRS document, which limits the generalizability of the findings. The model version temporality suggests that the findings represent the contemporary capabilities of the AI models, not permanent features. The evaluation's subjectivity, despite the use of systematic rubrics, and the variability in human baseline quality across different experience levels and organisational contexts also present limitations.

Research Question 5: How can the findings inform the development of an automated testing system?

The findings support a hybrid AI-human workflow rather than full automation. A three-phase approach that combines complementary strengths is optimal.

Phase 1: **AI-Driven Draft Generation** – AI models generate comprehensive baseline test suites with perfect requirements traceability, measurable test oracles, and systematic coverage of stated requirements.

Phase 2: **Human Expert Validation and Enhancement** – Professional QA engineers review AI-generated test cases, applying contextual knowledge to fill gaps, adjust risk priorities, and ensure environment-specific scenarios are included.

Phase 3: **AI-Assisted Analysis and Maintenance** – AI models check for consistency across enhanced suites, suggest additional scenarios, maintain traceability as requirements evolve, and recommend regression test selection.

This hybrid approach achieves 96% accuracy, 95% coverage, less than 1% hallucination rate, and 8× efficiency improvement compared to fully human workflows, while still benefiting from human judgment. For successful adoption, organisations should prioritise high-quality requirement documentation, establish appropriate human oversight, invest in prompt engineering, develop hallucination detection protocols, and implement phased rollouts to build trust and improve AI performance.

7.3 Research Contributions

This thesis provides several contributions to software engineering research and practice:

Empirical Evidence Base: This research establishes a comprehensive empirical foundation for understanding contemporary AI test generation capabilities through a systematic evaluation that incorporates rigorous methodology, statistical validation, and reproducible procedures. The findings extend beyond anecdotal observations, providing evidence-based assessments with appropriate confidence measures, significance testing, and reliability analysis, thereby ensuring the results are robust and credible.

Progressive Evaluation Methodology: The study's three-phase progressive structure, comprising requirements extraction, test generation comparison, and human vs. AI evaluation, provides a replicable framework for systematically assessing AI capabilities. This methodology strikes a balance between breadth (evaluating multiple models across varied scenarios) and depth (providing a detailed human comparison with controlled refinements). The design maintains scientific rigour, ensuring that the evaluation covers a wide range of conditions while offering in-depth insights into AI performance.

Dual-Perspective Analytical Approach: By combining independent researcher evaluation with LLM-assisted verification, the research introduces a novel quality assessment methodology. This approach acknowledges the value of both human expertise and AI analytical capabilities. The resulting inter-rater reliability ($= 0.72$) demonstrates substantial agreement between human evaluators, ensuring comprehensive and consistent evaluation coverage while validating the effectiveness of AI in real-world testing scenarios.

Detailed Limitation Characterisation: Rather than offering either uncritical enthusiasm or dismissive scepticism, the research provides a nuanced understanding of AI's strengths (e.g., systematic coverage, traceability, efficiency, and oracle quality) and shortcomings (e.g., contextual understanding, risk prioritisation, implicit requirements, and integration awareness). This balanced characterisation equips organisations with the knowledge needed to make informed adoption decisions, understanding both the potential and the limitations of AI in the testing process.

Practical Hybrid Workflow Model: The study presents a practical hybrid workflow model that combines AI and human collaboration, offering actionable guidance for organisations considering AI adoption. The evidence supporting this model achieving 96% accuracy, 95% coverage, and an 8-fold efficiency improvement sets realistic expectations regarding the benefits AI can bring, while emphasising the necessary human involvement to ensure quality and contextual relevance in the final outputs.

Hallucination Mitigation Strategies: The controlled refinements implemented in Experiments 9-15 demonstrated effective strategies for mitigating hallucinations, including the use of grounding constraints, RTM-first templates, policy-citation rules, and oracle contract schemas. These techniques provide a starting point for practitioners to develop prompt engineering strategies that can help reduce hallucinations and improve the quality of AI-generated test cases, contributing to more reliable AI test generation in future applications.

7.4 Practical Recommendations

Based on research findings, several recommendations guide organizational AI testing tool adoption:

7.4.1 For Organisations Considering AI Test Generation

Organisations should start by implementing **pilot projects in low-risk areas** to build organisational learning and trust before rolling out AI test generation across the entire enterprise.

- **High-quality requirement documentation** is crucial, as AI models perform best when specifications are comprehensive and detailed. With poor documentation, AI struggles to generate accurate tests.
- Establish clear **human oversight protocols** that strike a balance between validation thoroughness and efficiency.
- Expertise in **prompt engineering** should be developed through training and experimentation, as the quality of AI output depends heavily on the sophistication of the input.

- Plan for a 3–5× **net productivity improvement**, accounting for the validation overhead required for AI-generated tests, rather than expecting a full 10× theoretical speedup.
- Address team concerns by engaging in transparent communication, offering reskilling opportunities, and clearly defining how roles will evolve in tandem with AI adoption.

7.4.2 For QA Professionals

QA professionals should adapt their skills to focus on:

- Validating AI output, identifying contextual gaps, engineering prompts, and orchestrating human-AI workflow.
- Viewing AI as a tool for **augmentation rather than a replacement**, professional expertise remains crucial for contextual understanding, business judgment, risk assessment, and recognising implicit requirements.
- Developing capabilities in areas such as **hallucination detection**, business logic validation, and requirement completeness assessment.

QA professionals can position themselves as higher-value contributors by focusing on test strategy, exploratory testing, and quality architecture, rather than merely generating mechanical test cases.

7.4.3 For Tool Developers

Tool developers should focus on the following priorities:

- Prioritise **hallucination reduction** through improved grounding mechanisms and enhance contextual understanding (recognising implicit requirements and interpreting business logic).
- Develop **risk prioritisation features** that incorporate business context, rather than applying rigid, systematic rules.
- Improve **integration testing awareness** and multi-module validation capabilities.

- Work on creating **explainability features** that help users understand the AI's reasoning and identify potential errors.

Ultimately, designing **human-in-the-loop interaction patterns** is crucial, as it enables effective validation workflows and supports informed human decision-making throughout the process.

7.4.4 For Researchers

Researchers should expand their evaluation efforts:

- Include **multiple SRS documents** across various domains, complexity levels, and documentation qualities.
- Conduct **longitudinal studies** that assess the maintainability of AI test suites over the course of a project lifecycle.
- Investigate **advanced AI architectures** (e.g., multi-agent systems, retrieval-augmented generation, specialised fine-tuning).
- Develop **improved evaluation metrics** that capture nuanced quality dimensions beyond traditional measures of coverage and correctness.
- Study **organisational adoption patterns**, identifying success factors and failure modes in real-world AI production deployments.

7.4.5 Custom Model Development and Fine-Tuning Strategy

The research findings point toward custom fine-tuned models as the most promising path to overcoming identified limitations, particularly training data quality constraints and domain knowledge gaps.

Fine-Tuning Value Proposition

Organizations that have well-established software engineering practices and extensive internal documentation possess a valuable asset: high-quality proprietary training data. By using this data, such as internal SRS documents, API specifications, database schemas, test case

repositories, and bug tracking histories, organizations can fine-tune AI models to achieve significantly better performance compared to using general-purpose models.

Estimated Impact

Based on the performance gaps observed between AI-generated test cases and those created by human testers, especially the 14.3-point deficit in completeness and limitations in contextual understanding, a well-executed fine-tuning program could lead to the following improvements:

- Reduce hallucination rates (i.e., generating incorrect or made-up information) from 0.55 to less than 0.2 per 100 test steps.
- Investigate **advanced AI architectures** (e.g., multi-agent systems, retrieval-augmented generation, specialised fine-tuning).
- Increase recognition of implicit requirements from 72.8% to 85% or more.
- Improve risk prioritization agreement (measured by Cohen's kappa) from 0.34 to 0.60+.
- Achieve near-human completeness (85%) while still benefiting from AI's efficiency.

Organizational Prerequisites

To successfully fine-tune AI models, organizations must meet certain requirements:

1. **Data Volume**: You'll need a minimum of 500-1,000 high-quality SRS documents with corresponding test cases to train the model effectively. Larger organisations with years of project experience will likely have sufficient data, while smaller organisations may need to collaborate with others to gather the necessary data.
2. **Data Quality**: The training data must reflect current best practices. It shouldn't contain outdated methodologies or deprecated

technologies. Data curation and annotation should be done with the involvement of senior engineers to ensure their accuracy and relevance.

3. **Technical Infrastructure**: Fine-tuning large language models requires substantial computational resources (e.g., GPU clusters, high-memory systems) and specialised machine learning expertise. Organisations may use cloud-based fine-tuning services (such as OpenAI's custom model program or Anthropic's future services) to lower entry barriers.

4. **Continuous Update Cycles**: Since technology evolves rapidly, the fine-tuned models will need periodic retraining to incorporate new frameworks, security practices, and regulatory requirements.

7.4.6 Practical Implementation Approach

Organizations should approach fine-tuning in phases to ensure a smooth implementation:

1. **Phase 1 - Proof of Concept (3-6 months):**
    - Select 50-100 representative SRS documents from various projects.
    - Pair each SRS with **human-authored test suites**.
    - Fine-tune a smaller, open-source model (like Llama 3 70B) for cost-effective validation.
    - Evaluate the model's performance against a test set to measure improvements over the baseline model.

2. **Phase 2 - Production Pilot (6-12 months):**
    - Expand the training corpus to **500+ documents**.
    - Include **API documentation**, **database schemas**, and **architecture diagrams** (with manual annotations for diagrams).
    - Fine-tune a production-scale model (such as GPT-4 or Claude Opus).

- Deploy the model to a **single project team** for real-world validation.

3. **Phase 3 - Enterprise Rollout (12+ months):**
    - Incorporate feedback from the pilot phase, including edge cases.
    - Establish a **continuous learning pipeline** to integrate new projects and data.
    - Deploy the system **organization-wide**, with structured **human oversight protocols** to ensure quality and relevance.

Cost-Benefit Analysis

Fine-tuning requires a substantial investment:

- **Initial Development:** $100,000–$500,000 (for data curation, compute resources, and expertise).
- **Annual Maintenance:** $50,000–$150,000 (for retraining and infrastructure upkeep).

However, for **large organizations** that release **50+ software projects annually**, the efficiency gains from fine-tuning (estimated **8× net productivity improvement**) would translate to:

- **70–80% reduction** in test case generation time.
- Higher **coverage** and **consistency** across projects.
- **Freed capacity** for senior engineers to focus on strategic testing initiatives.

ROI Threshold: Organizations with **more than 50 QA engineers** generating **over 1,000 test cases annually** are likely to see a **positive ROI** within 18–24 months.

Alternative: Retrieval-Augmented Generation (RAG)

For organisations that cannot justify the investment in fine-tuning, Retrieval-Augmented Generation (RAG) offers a lighter alternative. RAG systems embed internal documents, like SRS, test case repositories, and technical specifications, into vector databases and retrieve relevant context for each AI query.

Advantages

- Lower implementation cost ($20,000–$100,000).
- No retraining required.
- Easier to **update** with new documents.

Limitations

- Less effective at learning organizational **style** and **terminology**.
- Depends on the quality of the **retrieved data**, meaning subtle contextual patterns may be missed.
- Typically achieves **50–70%** of the improvements that fine-tuning can provide.

7.5 Further Research Directions

This section outlines several potential avenues for future research based on the findings of this thesis. While this study has provided valuable insights into the role of generative AI in software testing, several areas warrant further exploration to enhance the capabilities of AI models and address current limitations. Below are some suggested directions for further research:

7.5.1 Improvement of Non-Functional Requirement Handling

The AI models in this research exhibited significant limitations in interpreting and generating test cases for non-functional requirements, including performance, security, and usability. Future research could

focus on developing specialised models or enhancing existing models to handle these complex requirements better. Specifically, research could explore how AI can be trained to generate comprehensive test cases for non-functional requirements that require specialised tools and testing environments.

Possible research questions:

- How can AI models be adapted to prioritize and generate test cases for non-functional requirements like load testing and security vulnerabilities?
- Can hybrid models (AI + human) optimize the testing process for non-functional requirements by leveraging human expertise in conjunction with AI's speed and consistency?

Contextual and Domain-Specific Knowledge

One of the key limitations identified in this study was AI's inability to fully comprehend domain-specific knowledge and business logic. This limitation is critical, particularly in fields such as healthcare, finance, or safety-critical systems where domain expertise is vital. Research could investigate how AI models can be augmented with domain-specific knowledge bases or integrated with expert systems to improve their contextual understanding.

Possible research questions

- How can domain-specific knowledge be integrated into AI models to improve their ability to generate contextually relevant test cases?
- What role can human experts play in validating or enriching AI-generated test cases for domain-sensitive systems?

Real-Time Adaptability and Learning

Another interesting direction is the real-time adaptability of AI in dynamic environments, such as Agile development. In Agile, requirements evolve rapidly, and test cases must be adjusted continuously.

Current AI models primarily work with static requirements documents, but future research could explore how AI can be trained to adapt to evolving specifications in real-time.

Possible research questions

- How can AI models be made more adaptive to continuous changes in requirements throughout an iterative software development process?
- What frameworks can be developed to enable AI models to learn from ongoing testing and incorporate feedback from real-world usage into their test case generation?

Human-AI Collaboration in Testing

While hybrid approaches (combining AI-generated test cases with human expertise) have shown promise, there is potential for deeper exploration into effective collaboration between human testers and AI systems. Research could examine methods for optimising human-AI workflows, ensuring that both AI's strengths in efficiency and human testers' strengths in creative problem-solving are maximised.

Possible research questions

- What are the most effective models for human-AI collaboration in software testing?
- How can AI-generated test cases be presented to human testers to facilitate quick and accurate validation without overwhelming them with redundant information?

Handling Ambiguity and Uncertainty

A significant challenge in software testing, both for AI and humans, is handling ambiguity in requirements. AI often generates test cases based on its understanding, which might not fully align with stakeholder intentions when the requirements are vague or incomplete. Future research could focus on enhancing AI's ability to identify ambiguities and request clarifications, similar to how human testers raise questions or flags during the testing phase.

Possible research questions

- How can AI models be improved to better handle ambiguous requirements by identifying uncertainty and requesting clarifications?
- Can AI models be designed to propose multiple test cases for ambiguous requirements, allowing human testers to choose the most appropriate one?

Long-Term Evaluation of AI-Generated Test Cases

While this study evaluated AI-generated test cases based on initial performance metrics, long-term effectiveness remains largely unexplored. Future research could investigate how AI-generated test cases perform over time, especially as software evolves. Does the accuracy of AI-generated test cases degrade as systems become more complex, or do they maintain their value through continuous refinement?

Possible research questions

- How do AI-generated test cases evolve over the lifecycle of a software product, and what strategies can be developed to ensure their continued relevance and accuracy?
- What is the long-term effectiveness of AI-generated test cases in detecting defects in evolving software systems?

Ethical Considerations and Accountability

As AI continues to play a larger role in software testing, ethical concerns related to AI-generated content become more important. These concerns include transparency, accountability, and the potential for biased test case generation. Research could explore how to mitigate ethical risks in AI-based testing by ensuring that AI systems are transparent, auditable, and accountable for their outputs.

Possible research questions

- How can ethical considerations be incorporated into the development of AI models for software testing?
- What frameworks can be established to ensure that AI models are transparent and accountable for their test case generation process?

Scaling AI Testing in Large-Scale Software Systems

While this study focused on a single case study, the scalability of AI testing tools in large, complex software systems remains an open question. Future work could examine how AI models perform when applied to large-scale, distributed systems with multiple teams working in parallel.

Possible research questions

- How can AI testing tools be scaled for use in large, multi-team software projects?
- What challenges arise when applying AI-based testing tools to systems with a high degree of complexity, and how can they be addressed?

7.6 Future Work

Several research directions emerge from this investigation's findings and limitations:

Multi-Domain Evaluation Expand the evaluation to include safety-critical systems (such as medical devices, aviation, and automotive), highly-regulated industries (like financial services, healthcare, and government), as well as embedded systems, real-time systems, and algorithm-intensive applications. These different areas may reveal unique AI performance patterns and challenges specific to each domain, which could necessitate specialized approaches to address them effectively.

Specification Quality Impact Analysis Examine how the quality of SRS documentation affects AI performance by systematically varying the quality from informal user stories to highly formal specifications. This study highlights how AI performance is sensitive to the quality of input documentation and clarifies the paradox where AI appears most valuable in scenarios where detailed specifications are often lacking, making AI tools a potential solution for addressing poorly defined requirements.

Longitudinal Maintenance Studies Track AI-generated test suites over multiple cycles of requirement changes, evaluating how the tests are maintained, how changes affect them, and how effective regression

testing is over time. It's essential to verify whether the initial quality of generated tests remains consistent as the system evolves, as long-term maintenance may pose challenges to AI-generated content.

Advanced Architecture Exploration Look into specialized approaches for improving AI performance in test generation, including methods like retrieval-augmented generation (which gives models access to relevant code, documentation, and historical test cases), multi-agent systems that use specialized agents for different types of testing (such as functional, security, performance, and integration testing), and fine-tuning approaches that train models on organization-specific codebases and testing practices. Additionally, consider hybrid symbolic-neural approaches that combine formal methods with the flexibility of large language models to improve the overall effectiveness of AI-generated tests.

Improved Risk Prioritization Develop new methods for incorporating business context, organizational priorities, and historical operational data into AI's risk assessment process. This could include learning from real-world production incident databases, stakeholder feedback, and economic impact models to help AI better prioritize test cases that address the most critical risks to the business.

Enhanced Evaluation Methodologies Create new evaluation metrics that measure coverage of implicit requirements, the appropriateness of business logic, and how well domain expertise is applied in AI-generated tests. Additionally, explore ways to automate hallucination detection and enhance techniques for inter-rater reliability to assess the subjective quality aspects of the generated tests.

Production Deployment Studies Conduct field studies in real-world organisational settings to measure the actual adoption success of AI test generation tools, looking at factors such as team satisfaction, productivity impacts, defect reduction, and total cost of ownership. These studies will help identify critical success factors and standard failure modes, providing insights that can guide adoption strategies in similar organisations.

Integration Testing Capabilities Explore ways to improve AI's ability to recognise system architecture, identify interactions between system components, and generate integration test scenarios. This is a significant gap in current AI test generation models, which often overlook the

complexity of multi-component systems and fail to create thorough integration tests.

Explainable Test Generation Develop methods to make AI-generated tests more explainable. This involves creating tools that enable AI models to explain their reasoning behind test generation decisions, such as how they interpret requirements, why specific tests are selected, and how coverage decisions are made. Increased explainability would facilitate human validation, build trust in AI-generated tests, and aid in debugging issues during the test generation process.

7.7 Final Remarks

This research demonstrates that contemporary large language models, particularly Claude, are capable of matching the accuracy of professional quality assurance engineers in structured test generation while excelling in areas such as requirements traceability, oracle measurability, and generation efficiency. However, AI still requires human oversight to ensure contextual completeness, alignment with business risk, and seamless organizational integration. The future of software testing does not lie in replacing humans but in fostering effective human-AI collaboration, where both parties leverage their complementary strengths.

The study provides a solid empirical foundation for organizations considering the adoption of AI testing tools, offering an evidence-based understanding that goes beyond the hype and scepticism often associated with AI. Organizations can realistically expect substantial productivity gains (a 3-5× net improvement) while maintaining or even improving test quality by implementing hybrid workflows. Achieving success requires realistic expectations, adequate preparation, and a commitment to preserving human expertise while integrating AI capabilities.

As large language models continue to evolve rapidly, the capabilities assessed in this research represent a current baseline, not a permanent limit. Future models may overcome the limitations discussed through architectural enhancements, specialized training, and advanced reasoning capabilities. However, the fundamental tension between AI's systematic processing and humans' contextual under-

standing suggests that hybrid approaches will likely remain the most effective strategy for the foreseeable future.

In conclusion, this research demonstrates that automated test generation has evolved from an academic curiosity to a practical reality. Organizations that invest in a thoughtful adoption process with the proper prerequisites, realistic expectations, and effective human-AI workflows can see significant improvements in quality assurance. The real challenge isn't whether AI can contribute to software testing, but rather how organizations can successfully integrate these capabilities while preserving the critical human expertise needed for testing.

Bibliography

[1] DOBSLAW, Felix; FELDT, Robert; YOON, Jeongju; YOO, Shin. Challenges in Testing Large Language Model-Based Software: A Faceted Taxonomy. 2025. Available from arXiv: 2503.00481.

[2] SPILLNER, Andreas; LINZ, Tilo; SCHAEFER, Hans. *Software Testing Foundations, 5th Edition: A Study Guide for the Certified Tester Exam*. Rocky Nook, 5th edition, 2021.

[3] AMMANN, Paul; OFFUTT, Jeff. *Introduction to Software Testing*. Cambridge University Press, 2016.

[4] PEZZÈ, Mauro; YOUNG, Michal. *Software Testing and Analysis: Process, Principles and Techniques*. Wiley, 2007.

[5] ALAMMAR, Jay; GROOTENDORST, Maarten. *Hands-On Large Language Models: Language Understanding and Generation*. O'Reilly, 2024.

[6] BERRYMAN, Jared; ZIEGLER, Albert. *Prompt Engineering for LLMs*. O'Reilly, 2024.

[7] NAVEED, Humza; KHAN, Asad Ullah; QIU, Shi; SAQIB, Muhammad; ANWAR, Saeed; USMAN, Muhammad; AKHTAR, Naveed; BARNES, Nick; MIAN, Ajmal. A Comprehensive Overview of Large Language Models. *ACM Transactions on Intelligent Systems and Technology*, 2025, vol. 16, no. 5, article 106.

[8] IEEE. *IEEE Standard for Software and System Test Documentation*. IEEE Std 829-2008, 2008.

[9] ISTQB. *ISTQB Certified Tester Foundation Level Syllabus*. International Software Testing Qualifications Board, version 4.0, 2023.

[10] WANG, Chunhui. Automatic Generation of Acceptance Test Cases From Use Case Specifications: An NLP-Based Approach. *IEEE Transactions on Software Engineering*, 2020, vol. 47, no. 11, pp. 2388–2407.

[11] LAFI, Mohammed; ALRAWASHED, Thamer; HAMMAD, Ahmad Munir. Automated Test Cases Generation From Requirements Specification. In: *Proc. International Conference on Information Technology* (*ICIT*). 2021, pp. 317–322.

[12] ALLALA, Sai Chaithra; SOTOMAYOR, Juan P.; SANTIAGO, Dionny; KING, Tariq M.; CLARKE, Peter J. Generating Abstract Test Cases from User Requirements using MDSE and NLP. In: *Proc. IEEE 22nd International Conference on Software Quality, Reliability and Security* (*QRS*). 2022, pp. 862–871.

[13] ARORA, Chetan; HERDA, Tomas; HOMM, Verena. Generating Test Scenarios from NL Requirements Using Retrieval-Augmented LLMs: An Industrial Study. In: *Proc. IEEE 32nd International Requirements Engineering Conference* (*RE*). 2024, pp. 123–133.

[14] HALDAR, Susmita; PIERCE, Mary; CAPRETZ, Luiz Fernando. Assessing the Effectiveness of ChatGPT in Preparatory Testing Activities. In: *Proc. IEEE Frontiers in Education Conference* (*FIE*). 2024, pp. 1–8.

[15] HALDAR, Susmita; PIERCE, Mary; CAPRETZ, Luiz Fernando. Exploring the Integration of Generative AI Tools in Software Testing Education: A Case Study on ChatGPT and Copilot for Preparatory Testing Artifacts in Postgraduate Learning. *IEEE Access*, 2025, vol. 13, pp. 1234–1248.

[16] CHAUHAN, Pooja; BALAKRISHNAN, Vijayakumar. Domain-Specific Software System Testing by Using Intelligent Approaches. In: *Proc. International Conference on Computational Intelligence and Network Systems* (*CINS*). 2024, pp. 45–52.

[17] AARIFEEN, Hafsa; WICKRAMAARACHCHI, Dilani. Evaluating AI-Based Unit Testing Tools: A Focus on Usability, Reliability, and Integration. In: *Proc. IEEE International Conference on Advanced Research in Computing* (*ICARC*). 2025, pp. 89–96.

[18] JIAO, Yuantao; WANG, Jian; LI, Runmei; WANG, Fei-Yue; ZHAO, Hongtao; XIONG, Gang. Intelligent Generation of Test Cases for

a Parallel Testing System: A Case Study on Railway Systems. *IEEE Intelligent Transportation Systems Magazine*, 2025, vol. 17, no. 2, pp. 78–92.

[19] CHENAIL-LARCHER, Zacharie; MINANI, Jean Baptiste; MOHA, Naouel. Test Generation from Use Case Specifications for IoT Systems: Custom, LLM-Based, and Hybrid Approaches. In: *Proc. IEEE International Conference on Software Testing, Verification and Validation* (*ICST*). 2025, pp. 156–167.

[20] HANDA, Devansh; NIKHIL, Kindi Krishna; KANAGARAJ, Kanaganandini; DUVARAKANATH, S.; SUPRIYA, M. Test Case Generation using Graph-based Deep Learning Methods. In: *Proc. IEEE International Conference on Artificial Intelligence and Machine Learning Applications* (*AIMLA*). 2025, pp. 234–241.

[21] OPENAI. GPT-4 Technical Report. 2023. Available from arXiv: 2303.08774.

[22] ANTHROPIC. Claude 3 Model Card. Technical Report, 2024. Available also from: `https://www.anthropic.com/claude`.

[23] MICROSOFT. Microsoft Copilot: Your AI companion. 2024. Available also from: `https://www.microsoft.com/copilot`.

[24] DEEPSEEK. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. Available from arXiv: 2405.04434.

[25] XAI. Grok-1: A Large Language Model. Technical Report, 2024. Available also from: `https://x.ai/`.

[26] BROWN, Tom et al. Language Models are Few-Shot Learners. In: *Advances in Neural Information Processing Systems 33* (*NeurIPS*). 2020, pp. 1877–1901.

[27] WEI, Jason et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: *Advances in Neural Information Processing Systems 35* (*NeurIPS*). 2022, pp. 24824–24837.

[28] YAO, Shunyu et al. ReAct: Synergizing Reasoning and Acting in Language Models. In: *Proc. International Conference on Learning Representations* (*ICLR*). 2023.

[29] LEWIS, Patrick et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems 33* (*NeurIPS*). 2020, pp. 9459–9474.

[30] ACHIAM, Josh et al. GPT-4 Technical Report. 2023. Available from arXiv: 2303.08774.

[31] ANIL, Rohan et al. PaLM 2 Technical Report. 2023. Available from arXiv: 2305.10403.

[32] TOUVRON, Hugo et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. 2023. Available from arXiv: 2307.09288.

[33] TEAM, Gemini et al. Gemini: A Family of Highly Capable Multimodal Models. 2023. Available from arXiv: 2312.11805.

[34] VASWANI, Ashish et al. Attention is All You Need. In: *Advances in Neural Information Processing Systems 30* (*NeurIPS*). 2017, pp. 5998–6008.

[35] DEVLIN, Jacob; CHANG, Ming-Wei; LEE, Kenton; TOUTANOVA, Kristina. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: *Proc. Conference of the North American Chapter of the Association for Computational Linguistics* (*NAACL*). 2019, pp. 4171–4186.

[36] RAFFEL, Colin et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 2020, vol. 21, no. 140, pp. 1–67.

[37] CLARK, Kevin; LUONG, Minh-Thang; LE, Quoc V.; MANNING, Christopher D. ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. In: *Proc. International Conference on Learning Representations* (*ICLR*). 2020.

[38] RADFORD, Alec et al. Learning Transferable Visual Models From Natural Language Supervision. In: *Proc. International Conference on Machine Learning* (*ICML*). 2021, pp. 8748–8763.

[39] HOFFMANN, Jordan et al. Training Compute-Optimal Large Language Models. 2022. Available from arXiv: 2203.15556.

[40] BISK, Yonatan et al. Experience Grounds Language. In: *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, pp. 8718–8735.

[41] BUBECK, Sébastien et al. Sparks of Artificial General Intelligence: Early experiments with GPT-4. 2023. Available from arXiv: 2303.12712.

[42] ZHANG, Jie; ZHAO, Junjie; ZHENG, Hanjie; TAN, Chengcheng. LLMs in Software Engineering: A Survey. 2023. Available from arXiv: 2312.15223.

[43] HOU, Xinyi et al. Large Language Models for Software Engineering: A Systematic Literature Review. 2023. Available from arXiv: 2308.10620.

[44] FAN, Yangrui et al. Large Language Models for Software Engineering: Survey and Open Problems. In: *Proc. IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE)*. 2024, pp. 31–38.

[45] CHEN, Mark et al. Evaluating Large Language Models Trained on Code. 2021. Available from arXiv: 2107.03374.

[46] JIANG, Nan et al. Impact of Code Language Models on Automated Program Repair. In: *Proc. IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2023, pp. 1430–1442.

[47] XIA, Chunqiu Steven; ZHANG, Lingming. Keep the Conversation Going: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. 2023. Available from arXiv: 2304.00385.

[48] PEARCE, Hammond et al. Examining Zero-Shot Vulnerability Repair with Large Language Models. In: *Proc. IEEE Symposium on Security and Privacy (S&P)*. 2023, pp. 2339–2356.

[49] PRENNER, Julian Aron; ROBBES, Romain. Automatic Program Repair with OpenAI's Codex: Evaluating QuixBugs. 2022. Available from arXiv: 2111.03922.

[50] JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? 2023. Available from arXiv: 2310.06770.

A An appendix

All experimental data, prompts, analysis results, and supplementary materials for this research are publicly available in the following GitHub repository:

Repository: `https://shorturl.at/CbhIw`

The repository contains:

- Complete experimental data from all 32 experiments (7 requirement extraction + 25 test generation experiments across three phases)
- AI-generated test cases and test management plans from all five models (ChatGPT, Microsoft Copilot, DeepSeek, Grok, Claude)
- Human-written test cases baseline (JSON format)
- All prompts and prompt templates used throughout the research
- Source requirements documents (SRS, user stories, use cases)
- Statistical analysis calculations and supplementary data
- Research instruments (evaluation criteria, scoring rubrics)

The repository is organized with clear documentation to facilitate reproducibility and verification of the research findings.